



Rapport de Stage d'Été

Présenté à :

École Nationale d'Ingénieurs de Sfax

Département de Génie Informatique

Organisme d'accueil :



Sujet :

***Agent RAG pour l'automatisation du traitement et de
la digitalisation des factures et devis***

Réalisé par :

Siwar ADALI

Encadrant académique : Sahla MASMOUDI

Encadrant professionnel : Youssef BOUATTOR

Année Universitaire : 2025/2026

Remerciements

Je tiens à remercier toute l'équipe pour son accueil chaleureux, sa disponibilité et les conditions favorables qu'elle m'a offertes durant ce stage. J'adresse une reconnaissance particulière à **M. Youssef Bouattour** pour son accompagnement, ses précieux conseils, ses retours constructifs et la confiance qu'il m'a accordée. Son soutien a largement contribué à la réussite de cette expérience.

J'adresse mes sincères remerciements à mon encadrante académique, **Mme Sahla Masmoudi**, pour son accompagnement et ses conseils tout au long de ce projet. Je remercie également l'ensemble de mes enseignants à l'École Nationale d'Ingénieurs de Sfax pour les connaissances et les bases méthodologiques qu'ils m'ont transmises, et qui m'ont permis d'aborder ce travail avec rigueur et confiance.

Je leur adresse mes remerciements les plus sincères pour leur soutien, leur patience et leurs encouragements constants. Leur présence à mes côtés a été une source inestimable de motivation tout au long de ce parcours.

À toutes celles et ceux qui ont contribué à cette aventure, merci.

Contents

Remerciements	I
Introduction	1
1 Notions théoriques	2
1.1 Traitement intelligent de documents	2
1.1.1 Les étapes d'un pipeline IDP	2
1.1.2 Application au projet	3
1.2 Recherche augmentée par génération	4
1.2.1 Représentation vectorielle du contenu	4
1.3 Recherche hybride et <i>grounding</i>	5
1.3.1 Fusion des classements par RRF	5
1.3.2 Le principe de grounding	5
1.4 Synthèse	6
2 Démarche et méthodologie	7
2.1 Objectif de comparaison	7
2.2 Critères utilisés	7
2.3 Analyse des approches	8
2.3.1 Power Automate	8
2.3.2 AI Builder	8
2.3.3 Azure Document Intelligence	10
2.3.4 Bilan de la comparaison	10
2.4 Choix méthodologique	10
2.4.1 Validation de la prise en charge des devis	11
2.5 Organisation du travail et suivi	14

3	Réalisation du pipeline IDP	15
3.1	Vue d'ensemble des sprints 1 à 6	15
3.2	Extraction et validation	17
3.2.1	Alerte automatique sur anomalie critique	17
3.2.2	Cohérence de l'extraction sur des doublons	18
3.2.3	Vérification croisée du journal d'erreurs	19
3.3	Modèle de données Postgres	20
3.4	Génération des embeddings	21
3.5	Moteur de recherche hybride	24
3.6	Vérification et validation du classement	26
3.7	Bilan du pipeline IDP	30
4	Intégration avec l'outil de facturation	31
4.1	Objectif du sprint 7	31
4.2	Synchronisation Postgres vers Dataverse	32
4.3	Formulaire Power Apps	34
4.4	Synchronisation du paiement	36
4.5	Bugs résolus pendant l'intégration	37
5	Réalisation de l'agent RAG final	38
5.1	Refactoring modulaire	38
5.2	Détection d'intention et routage	39
5.3	Génération de réponses groundées	39
5.4	Reporting automatique	40
5.5	Orchestration globale	40
6	Difficultés rencontrées et solutions apportées	41
6.1	Quota Gemini dépassé	41
6.2	Dépréciation du modèle	41
6.3	Problèmes de types numériques	41
6.4	Paramètres SQL dans les CTE	42
6.5	Erreurs Dataverse et Power Automate	42

7 Résultats et évaluation	43
7.1 Jeu de documents traité	43
7.2 Exemples d'interrogation de l'agent	43
7.3 Rapports générés	44
7.3.1 Rapport 1 — Montant TTC par fournisseur	45
7.3.2 Rapport 2 — Reste à payer par statut	47
7.3.3 Rapport 3 — Nombre de documents par mois	49
7.4 Traçabilité	51
7.5 Évaluation globale	52
8 Limites identifiées et recommandations	54
8.1 Limites	54
8.1.1 Qualité variable des documents sources	54
8.1.2 Dépendance aux services externes	54
8.1.3 Contraintes de la plateforme Power Platform	55
8.1.4 Absence de déduplication effective	55
8.1.5 Détection d'intention par mots-clés	55
8.2 Recommandations	55
8.2.1 Tableau de bord de supervision	56
8.2.2 Renforcement de la validation métier	56
8.2.3 Transparence des sources citées	57
A Annexes techniques	58
A.1 Organisation du jeu documentaire et du stockage	58
A.2 Schéma complet de la table <code>document</code>	59
A.3 Tables de la base	59
A.4 Modules Python de l'agent final	60
Conclusion et apprentissages personnels	61
Résumés / Abstracts	62

List of Figures

1.1	Étapes d'un pipeline IDP	3
1.2	Principe du RAG : récupération puis génération	4
1.3	Synthèse de la chaîne IDP puis RAG	6
2.1	Échec AI Builder par capacité de crédit épuisée	9
2.2	Grille comparative des trois approches d'extraction	10
2.3	Premier devis utilisé pour le test manuel	12
2.4	Premier test manuel AI Builder sur un devis	13
2.5	Second test manuel AI Builder sur un devis différent	13
2.6	Suivi du temps par tâche	14
3.1	Flux Power Automate final après intégration Postgres	16
3.2	Alerte email sur anomalie critique	18
3.3	Test de cohérence de l'extraction sur des doublons	19
3.4	Export Excel des erreurs de traitement	19
3.5	Table journal_erreurs dans Neon	20
3.6	Tables de la base PostgreSQL dans Neon	20
3.7	Documents insérés, vus depuis la table client	21
3.8	Génération d'un embedding	22
3.9	Récapitulatif final des embeddings insérés	23
3.10	Vérification des dimensions des embeddings dans Neon	23
3.11	Exemples de questions adressées au moteur	24
3.12	Exemple de recherche hybride, premier cas	25
3.13	Exemple de recherche hybride, second cas	26

3.14	Vérification croisée dans Neon, premier contrôle	27
3.15	Vérification croisée dans Neon, second contrôle	27
3.16	Vérification du filtre fournisseur	28
3.17	Test de l'agrégation sur un montant total	29
3.18	Test du dénombrement des anomalies	29
3.19	Test de l'agrégation filtrée par fournisseur	30
4.1	Architecture d'intégration	31
4.2	Prolongement du flux vers Dataverse	32
4.3	Structure de la table Document Dataverse	33
4.4	Table Documents Dataverse peuplée	34
4.5	Vue « Documents actifs » de l'outil de facturation	35
4.6	Détail d'un document dans le formulaire	35
4.7	Flux de synchronisation du paiement	36
4.8	Table document synchronisée dans Neon	37
5.1	Pipeline d'interrogation de l'agent RAG	38
7.1	Test simple de l'agent	44
7.2	Filtre fournisseur et agrégation	44
7.3	Rapport 1, montant TTC par fournisseur	45
7.4	Rapport 1, détail des données	46
7.5	Rapport 2, reste à payer par statut	47
7.6	Rapport 2, graphique et détail	48
7.7	Rapport 3, nombre de documents par mois	49
7.8	Rapport 3, détail par mois	50
7.9	Graphique de synthèse	51
7.10	Organisation par timestamp	51
7.11	Rapport supplémentaire	52
A.1	Organisation documentaire dans SharePoint	58

Introduction

Dans un contexte de transformation numerique, les entreprises Une attention particuliere est egalement accordee a la qualite des donnees, a la maîtrise des couts, a la fiabilite du systeme et a l'experience utilisateur. traitent quotidiennement un volume croissant de documents administratifs, notamment des factures et des devis. Ces documents contiennent des informations essentielles, telles que les coordonnees des fournisseurs, les montants, les taxes et les dates d'echeance. Leur traitement manuel demeure toutefois long, repetitif et expose aux erreurs de saisie, de vérification et de classement.

C'est dans cette perspective que s'inscrit mon stage d'ete realise au sein de Dazzlifai, autour du sujet intitule : *Agent RAG pour l'automatisation du traitement et de la digitalisation des factures et devis*. Ce projet repond a une problematique principale : comment automatiser l'exploitation de documents heterogenes tout en garantissant la fiabilite, la tracabilite et l'accessibilité des données extraites ?

L'objectif consiste a concevoir une solution capable de recevoir des factures et des devis, d'en extraire les informations pertinentes, de les valider, puis de les stocker dans une base de données structurée. La solution permet également à l'utilisateur d'interroger ces données en langage naturel afin d'obtenir des reponses precises et fondees sur le contenu des documents traités. Pour atteindre cet objectif, une chaine de traitement complete a ete mise en place. Elle combine le traitement intelligent de documents, une base de donnees PostgreSQL, des representations vectorielles, une recherche hybride et un mecanisme de generation augmentee par la recherche (RAG). Son intégration avec Microsoft Power Platform facilite l'automatisation des flux de travail et l'interaction avec les utilisateurs.

Ce rapport retrace les differentes etapes de conception et de realisation de cette solution. Le premier chapitre presente le contexte du projet ainsi que les notions theoriques essentielles, notamment le traitement intelligent de documents (IDP) et la generation augmentee par la recherche (RAG). Le deuxieme chapitre expose l'etude comparative menee entre Power Automate, AI Builder et Azure Document Intelligence, ainsi que les choix technologiques retenus. Les chapitres suivants detaillent la realisation du projet sprint par sprint, les difficultés rencontrées, les solutions apportees et les resultats obtenus. Enfin, le rapport presente les limites de la solution, ses perspectives d'amelioration et les principaux acquis techniques et personnels issus de cette expérience.

Chapter 1

Notions théoriques

Ce chapitre présente les deux notions théoriques centrales du projet : le traitement intelligent de documents (IDP) et la génération augmentée par récupération (RAG). Ces deux notions ne sont pas indépendantes : la première alimente la base de connaissances que la seconde vient interroger. Les chapitres 3 et 5 montrent comment elles s’articulent concrètement dans la solution développée durant le stage.

1.1 | Traitement intelligent de documents

Le traitement intelligent de documents, ou IDP, désigne l’ensemble des techniques qui permettent de transformer un document non directement exploitable par un système informatique en données structurées. Dans le cas d’une facture, le document peut être un PDF natif, une image scannée ou une simple photo prise avec un téléphone. Pour un être humain, lire le nom du fournisseur, la date d’émission ou le montant total est immédiat. Pour une application, cela suppose une succession d’étapes : reconnaître le texte présent dans l’image, comprendre la disposition du document, associer chaque libellé à la bonne valeur, puis vérifier que le résultat obtenu est cohérent.

1.1.1 Les étapes d’un pipeline IDP

Un pipeline IDP typique s’organise en quatre étapes successives, illustrées dans la figure 1.1. Chacune transforme progressivement le document brut en informations structurées et directement exploitables par le système. Ces étapes sont étroitement liées, car la qualité du résultat obtenu à chaque niveau influence la fiabilité de l’ensemble du processus :

1. **La reconnaissance optique de caractères (OCR)**, qui convertit les zones visuelles du document en texte brut. Cette étape est nécessaire mais rarement suffisante : une

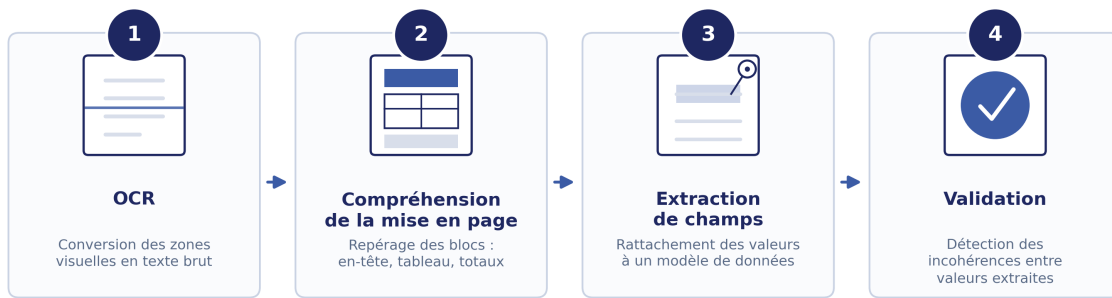


Figure 1.1: Les quatre étapes successives d'un pipeline de traitement intelligent de documents.

facture n'est pas une simple suite de mots, mais un document structuré comprenant un en-tête, un tableau de lignes de détail, des totaux, des taxes et des blocs d'identification du client et du fournisseur.

2. **La compréhension de la mise en page**, qui consiste à repérer ces blocs structurels indépendamment du texte qu'ils contiennent, afin de savoir, par exemple, que tel nombre appartient au bloc des totaux plutôt qu'à une ligne de détail.
3. **L'extraction de champs**, qui rattache chaque valeur repérée à un champ précis d'un modèle de données : numéro de document, dates d'émission et d'échéance, montants HT/TVA/TTC, devise, identité des parties, lignes de détail.
4. **La validation**, qui confronte les valeurs extraites entre elles pour détecter des incohérences — par exemple un montant TTC qui ne correspond pas à la somme du montant HT et de la TVA annoncée, ou une date d'échéance antérieure à la date d'émission.

1.1.2 Application au projet

Dans ce stage, l'IDP a été utilisé pour automatiser la lecture des factures et des devis reçus par l'entreprise. Les informations extraites ont ensuite été validées, enrichies, puis stockées dans une base PostgreSQL. L'intérêt principal de cette approche ne réside pas seulement dans le gain de temps par rapport à une saisie manuelle, mais surtout dans la possibilité de concentrer la vérification humaine sur les documents réellement incertains. Pour cette raison, chaque document conserve, en plus des champs extraits, des scores de confiance, notamment sur le montant et l'identification du fournisseur, ainsi qu'un statut d'anomalie explicite lorsque les vérifications de cohérence échouent.

1.2 | Recherche augmentée par génération

Le RAG, ou *Retrieval-Augmented Generation*, peut se comprendre comme une façon de donner une mémoire fiable et vérifiable à un modèle de langage. Un grand modèle de langage (LLM) peut produire une réponse fluide et grammaticalement correcte, mais il ne connaît pas les données internes de l'entreprise : il n'a, par construction, jamais vu les factures traitées. Livré à lui-même face à une question précise sur ces documents, il risque de générer une réponse plausible, mais incorrecte ; ce phénomène est appelé *hallucination*.

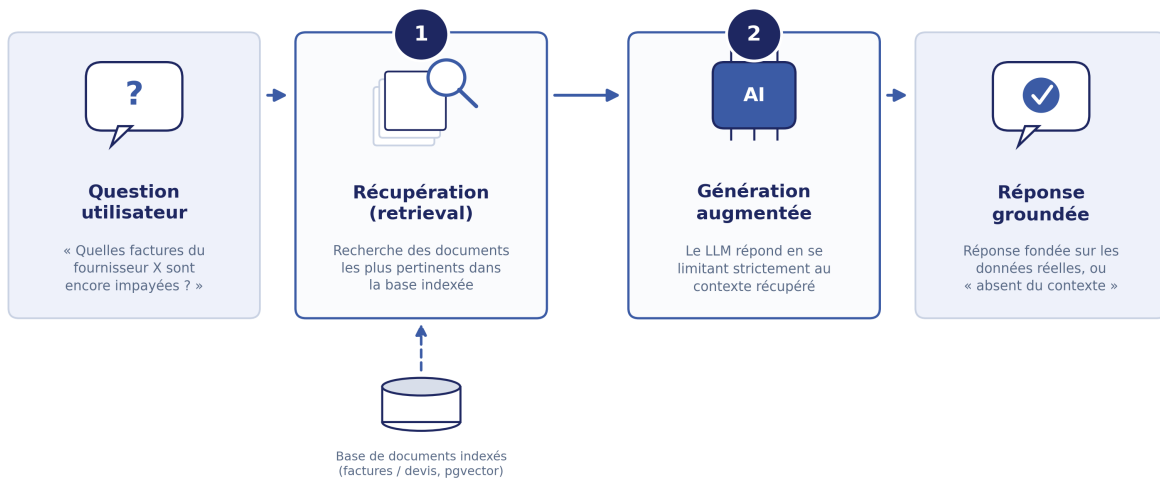


Figure 1.2: Les deux temps de l'approche RAG : récupération des documents pertinents, puis génération d'une réponse strictement fondée sur ce contexte.

L'approche RAG découpe le problème en deux temps, illustrés sur la figure 1.2 : le système commence par rechercher les documents ou les enregistrements pertinents par rapport à la question posée, puis demande au modèle de générer sa réponse en se limitant strictement aux éléments récupérés. Une analogie simple consiste à imaginer un assistant qui commence toujours par consulter les documents appropriés avant de formuler sa réponse. Si l'information recherchée n'y figure pas, il doit l'indiquer explicitement plutôt que de la deviner. C'est précisément le comportement attendu du système développé durant le stage.

1.2.1 Représentation vectorielle du contenu

Concrètement, le RAG mis en œuvre repose sur la notion d'*embedding* : une représentation numérique du contenu textuel sous la forme d'un vecteur de grande dimension (1536 dimensions dans ce projet). Cette représentation est construite de manière à ce que deux textes sémantiquement proches produisent des vecteurs voisins, indépendamment du vocabulaire exact employé. Chaque document est ainsi transformé en vecteur lors de son indexation, puis

stocké dans PostgreSQL grâce à l’extension `pgvector`, qui permet d’effectuer des recherches par similarité directement en SQL. Lorsqu’un utilisateur pose une question, celle-ci est transformée en vecteur selon le même procédé afin de retrouver les documents les plus proches à l’aide de la distance cosinus. La mise en œuvre concrète de cette étape, avec des exemples réels de vecteurs générés, est présentée au chapitre 3.

1.3 | Recherche hybride et *grounding*

La recherche vectorielle seule est efficace pour retrouver des documents proches par le sens, mais elle n’est pas toujours adaptée aux critères précis, tels qu’une année, un nom de fournisseur exact ou un statut de paiement : deux formulations sémantiquement proches peuvent désigner deux fournisseurs différents. Pour cette raison, le moteur développé combine deux approches complémentaires : une recherche vectorielle (par proximité sémantique) et une recherche textuelle classique (recherche plein texte PostgreSQL, basée sur la correspondance lexicale exacte).

1.3.1 Fusion des classements par RRF

Les résultats de ces deux recherches sont fusionnés à l’aide de la méthode *Reciprocal Rank Fusion* (RRF), qui attribue à chaque document un score dépendant de son *rang* dans chaque classement plutôt que de son score brut :

$$\text{score}_{\text{RRF}}(d) = \frac{1}{k + \text{rang}_{\text{vecteur}}(d)} + \frac{1}{k + \text{rang}_{\text{texte}}(d)}$$

avec, dans l’implémentation retenue, une constante $k = 60$. Cette formulation présente un double avantage : elle ne nécessite pas de normaliser des scores dont les échelles ne sont pas directement comparables (une distance cosinus et un score de pertinence textuelle n’ont pas la même unité), et elle privilégie naturellement les documents bien classés dans les *deux* recherches à la fois, plutôt qu’un document excellent dans une seule d’entre elles. Ce classement fusionné est en outre restreint, en amont, par des filtres structurés (année, statut, fournisseur, statut de paiement) détectés directement dans la question, ce qui évite de faire porter à la seule recherche sémantique des critères qu’une condition SQL classique traite de façon plus fiable.

1.3.2 Le principe de grounding

Le *grounding* des réponses désigne le fait que la génération du modèle de langage reste strictement rattachée aux données effectivement retrouvées par le moteur de recherche. Dans le cadre du stage, le modèle Gemini est utilisé pour rédiger la réponse finale présentée à l’utilisateur,

mais cette rédaction est contrainte, par une instruction explicite, à ne s'appuyer que sur les documents récupérés par le moteur RAG. Lorsque l'information demandée n'y figure pas, le modèle doit l'indiquer clairement plutôt que de la déduire ou de l'inventer. Ce principe est particulièrement important dans un contexte financier : un montant, une date d'échéance ou un statut de paiement erroné peut avoir des conséquences bien plus concrètes qu'une simple imprécision rédactionnelle.

1.4 | Synthèse

L'IDP et le RAG interviennent ainsi à deux moments distincts, mais complémentaires, de la même chaîne de traitement : le premier transforme des documents bruts en données fiables et structurées ; le second permet d'interroger ces données en langage naturel sans compromettre la fiabilité obtenue à l'étape précédente, comme le résume la figure 1.3.

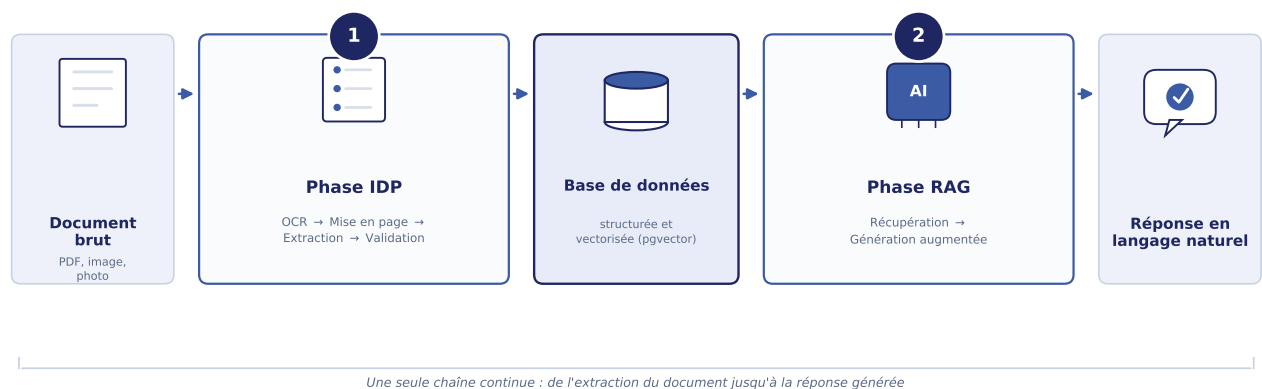


Figure 1.3: Continuité entre la phase d'IDP et la phase de RAG, articulées autour d'une base de données commune.

Cette continuité, de l'extraction jusqu'à la réponse générée, structure les réalisations présentées dans les chapitres 3 et 5.

Chapter 2

Démarche et méthodologie

2.1 | Objectif de comparaison

Avant de construire le pipeline final, une phase de comparaison a été nécessaire pour choisir l'approche d'extraction la plus adaptée. Cette étape correspond, dans le suivi Azure DevOps du stage, aux tâches 73 (choix et documentation du modèle d'extraction) et 74 (tests sur un échantillon varié de documents). Trois pistes ont été étudiées : Power Automate comme solution low-code d'orchestration, AI Builder comme service d'extraction intégré à Power Platform, et Azure Document Intelligence comme service spécialisé dans l'analyse de documents.

Le choix ne dépendait pas uniquement de la qualité brute de l'extraction. Il devait également tenir compte de l'intégration avec l'environnement cible retenu pour la suite du projet, notamment PostgreSQL sur Neon, Dataverse et Power Apps. D'autres critères ont été considérés, tels que le coût d'exploitation à long terme, la facilité de maintenance par une personne non spécialiste de l'apprentissage automatique (ML), la disponibilité des connecteurs et des accès durant le stage, ainsi que la capacité de chaque solution à fournir un score de confiance exploitable par le pipeline de validation.

2.2 | Critères utilisés

Les critères retenus, et la raison de leur choix, sont les suivants :

- **Fiabilité** : capacité à extraire correctement les champs importants comme le montant, la date, le fournisseur et les lignes de document, y compris sur des documents de qualité variable (scan incliné, faible résolution, mise en page non standard).
- **Coût** : impact des licences, limites d'utilisation et dépendance aux quotas — un critère qui s'est révélé particulièrement pertinent a posteriori, les quotas gratuits des services d'IA générative ayant effectivement été atteints plus tard dans le projet (voir chapitre 6).

- **Complexité d'intégration** : facilité de connexion avec PostgreSQL, puis avec Dataverse, Power Apps et les traitements Python développés en parallèle.
- **Maintenabilité** : possibilité de corriger les erreurs, surveiller le pipeline et faire évoluer le modèle sans dépendre d'une expertise pointue en science des données.
- **Contrôle technique** : niveau de liberté pour gérer les schémas de données, les erreurs, les scores de confiance et les traitements supplémentaires (par exemple les validations métier spécifiques à l'entreprise).

2.3 | Analyse des approches

2.3.1 Power Automate

Power Automate a été utilisé pour orchestrer les traitements et automatiser les synchronisations. Son principal avantage réside dans son intégration avec l'écosystème Microsoft déjà utilisé chez Dazzlifai. Il permet de créer rapidement des flux déclenchés par un événement, comme l'arrivée d'un nouveau document ou la modification d'un champ dans Power Apps, sans nécessiter d'infrastructure serveur dédiée. En revanche, Power Automate ne dispose pas, à lui seul, de fonctionnalités de compréhension documentaire : il agit principalement comme un orchestrateur connecté à un service d'extraction tiers.

2.3.2 AI Builder

AI Builder a apporté une solution pratique pour extraire les champs depuis les factures et devis, directement pilotable depuis un flux Power Automate. Son intérêt principal est d'être accessible dans une logique low-code et de s'intégrer naturellement avec Dataverse, ce qui simplifiait d'autant la suite du projet une fois l'intégration à l'outil de facturation devenue un objectif (voir chapitre 4).

Les tests réalisés ont cependant fait apparaître des points de vigilance concrets. La mise en route du modèle de reconnaissance de factures a nécessité la configuration d'un compte de stockage Azure dédié, une dépendance non immédiatement visible lors du choix initial de l'outil et qui a retardé la première itération de test. Ce type de contrainte d'infrastructure, propre au fonctionnement interne d'AI Builder, ne serait pas apparu dans une simple comparaison sur documentation : il n'a été identifié qu'au moment de la mise en œuvre réelle, ce qui confirme l'intérêt d'une phase de test effective plutôt que d'une comparaison purement théorique.

Un second blocage, plus limitant encore, est apparu lors de l'exécution répétée des flux de test dans l'environnement Power Platform fourni par l'École Nationale d'Ingénieurs de Sfax

(ENIS) : l'action AI Builder chargée de traiter chaque document échouait systématiquement avec le statut `Forbidden` (code HTTP 403), et le message suivant s'affichait au niveau du flux :

Échec de l'exécution du flux. Crédit Copilot ou capacité de crédit AI Builder manquant : Vous n'avez pas de capacité de crédit Copilot ni de crédit AI Builder dans cet environnement.

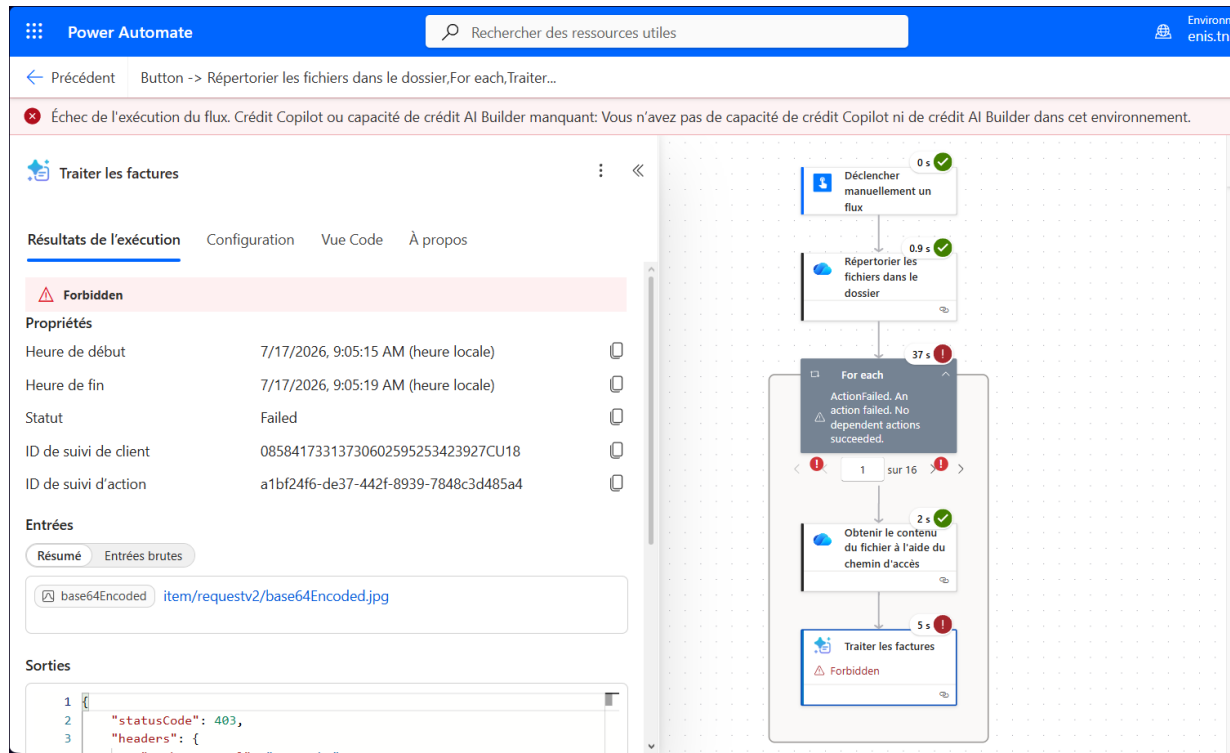


Figure 2.1: Échec de l'action AI Builder dans deux flux de test distincts, provoqué par l'absence de capacité de crédit dans l'environnement Power Platform de l'ENIS.

Contrairement aux difficultés documentées au chapitre 6, ce blocage ne provenait pas d'une erreur de conception du flux : la même erreur s'est reproduite à l'identique sur deux flux différents (`Traiter les factures` et `Document_processor_test`), avec un temps d'exécution de l'action de quelques secondes seulement avant l'échec, ce qui exclut un problème de logique métier ou de mapping de champs. Il s'agissait d'une limite de capacité propre à l'environnement Power Platform mutualisé fourni par l'établissement : les appels aux modèles AI Builder consomment un quota de crédits IA distinct des licences Power Automate elles-mêmes, quota qui n'était pas disponible dans cet environnement de test.

Cette contrainte a eu un impact direct sur la suite de la comparaison : elle confirme empiriquement le critère *Coût* déjà identifié au tableau 2.1, et illustre concrètement la différence entre un outil *disponible* et un outil *exploitable dans le contexte réel du stage* : une solution ne peut être retenue que si l'environnement mis à disposition permet effectivement de l'exécuter de bout en bout, indépendamment de ses qualités techniques intrinsèques.

2.3.3 Azure Document Intelligence

Azure Document Intelligence a été étudié comme solution plus spécialisée pour l'analyse documentaire. Elle offre une approche robuste pour les documents structurés et semi-structurés, avec un contrôle plus fin sur le modèle utilisé (modèle générique de factures ou modèle personnalisé entraîné sur des documents spécifiques à l'entreprise). Son mode d'intégration diffère cependant fondamentalement de celui d'AI Builder : l'appel s'effectue par l'intermédiaire d'une API REST intégrée directement dans le code Python du pipeline, plutôt que par un connecteur low-code. Cette approche demande davantage d'efforts lors de l'intégration initiale, mais offre un contrôle plus complet sur le traitement des réponses et des erreurs.

2.3.4 Bilan de la comparaison

Le tableau 2.1 et la figure 2.2 synthétisent l'évaluation qualitative des trois approches sur les cinq critères retenus.

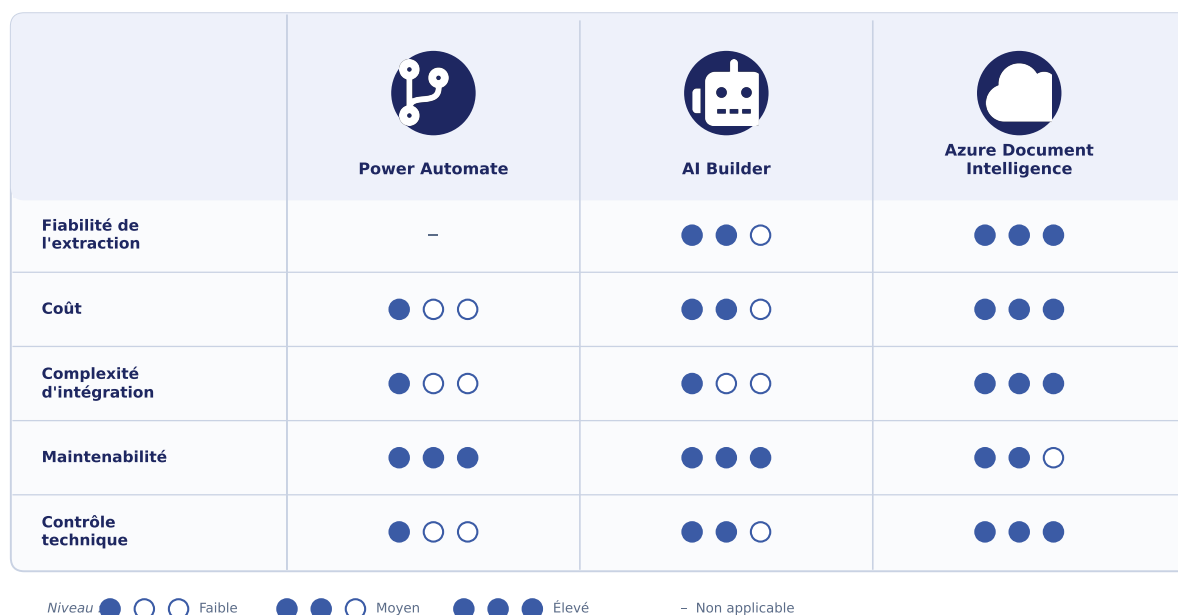


Figure 2.2: Comparaison visuelle des trois solutions évaluées sur les cinq critères retenus.

2.4 | Choix méthodologique

La démarche adoptée a consisté à combiner les avantages des outils plutôt qu'à rechercher une solution unique. Power Automate a été conservé pour l'orchestration et la synchronisation. AI Builder a été retenu pour l'extraction, principalement en raison de sa rapidité de mise en

Table 2.1: Synthèse comparative des trois approches d'extraction évaluées

Critère	Power Automate	AI Builder	Azure Doc. Intelligence
Fiabilité de l'extraction	N/A (orchestrateur)	Correcte	Élevée
Coût	Faible	Modéré (quotas)	Modéré à élevé
Complexité d'intégration	Faible	Faible (low-code)	Élevée (API)
Maintenabilité	Élevée	Élevée	Moyenne
Contrôle technique	Faible	Moyen	Élevé

œuvre en low-code, de son intégration native avec Dataverse et de sa capacité à traiter des factures comme des devis avec un même modèle prédéfini. Ce dernier critère était déterminant, puisque le périmètre du projet couvrait les deux types de documents. La base PostgreSQL hébergée sur Neon est devenue le point central pour stocker les données structurées, les statuts, les scores de confiance et, par la suite, les embeddings vectoriels.

2.4.1 Validation de la prise en charge des devis

Le modèle prédéfini de reconnaissance de documents d'AI Builder est présenté par Microsoft comme un modèle de factures. Avant de retenir définitivement cet outil pour l'ensemble du périmètre du projet, il fallait donc vérifier qu'il restait exploitable sur des devis, dont la structure est proche d'une facture (fournisseur, client, lignes de détail, montants) mais qui n'en partage pas toutes les mentions. Deux tests manuels ont été menés directement dans l'interface AI Builder, indépendamment du flux Power Automate, sur deux devis distincts.

Le premier test a porté sur un devis émis par l'entreprise fictive TEMPO Gestion de Patrimoine.

Le résultat confirme que le modèle prédéfini de factures reste exploitable sur ce devis : le nom du client (*Thomas Garcia*), son adresse et le numéro du document ont été correctement identifiés et rattachés aux bons champs.

Un second test, mené sur un devis totalement différent (émis par Lumière Studio Vidéo, à destination du client BioAlimentaire France), a également produit une extraction correcte : adresse de facturation, nom du client et dates ont été identifiés avec cohérence par rapport au document source.

Ces deux tests, menés sur des devis de structures et de contenus différents, ont conforté le choix d'AI Builder comme solution d'extraction pour l'ensemble du pipeline : un seul modèle suffisait à couvrir les factures et les devis traités par Dazzlifai, sans devoir maintenir deux

Devis n° 0123



Gestion de Patrimoine
123 Anywhere St., Any City
+123-456-7890
www.reallygreatsite.com

Client: Thomas Garcia
123 Anywhere St., Any City
+123-456-7890
www.reallygreatsite.com

Description	Prix unitaire	Quantité	Total HT
Conseil	500€	1	500€
Frais de dossier	100€	1	100€
Autres	50€	1	50€
Subtotal			650€
TVA (0%)			\$0
Total			650€

Informations de paiement
Paiement par virement bancaire
Compte : 0123 4567 8901

Termes & conditions
Ce devis est valable 1 mois à compter de sa date d'émission

Date: 7/28/2026

Signature: 

Figure 2.3: Devis n° 0123, client Thomas Garcia, utilisé pour le premier test manuel.

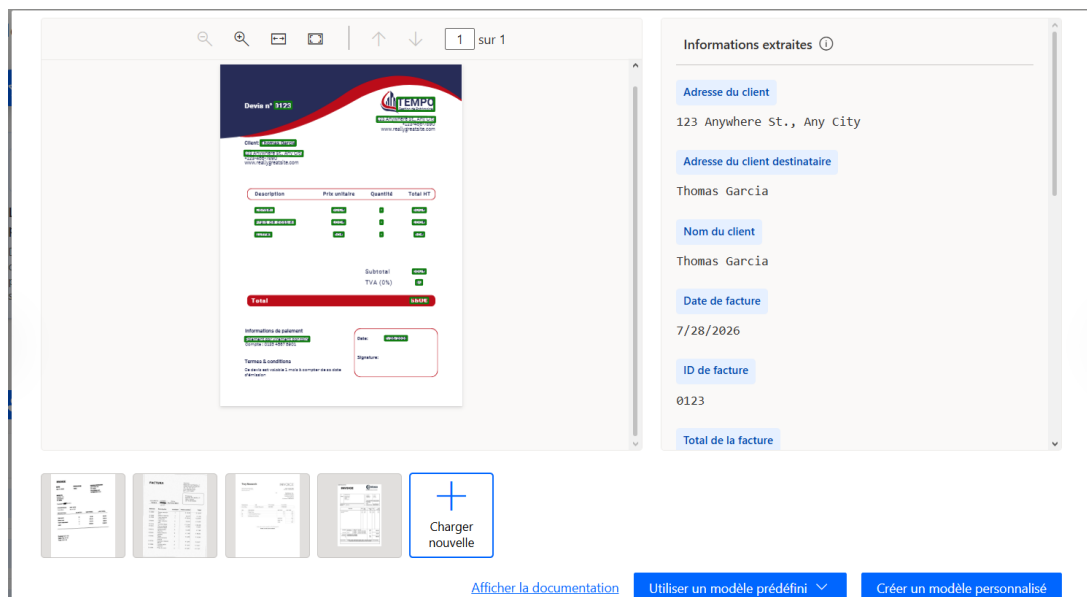


Figure 2.4: Extraction obtenue sur le premier devis testé : informations correctement identifiées.

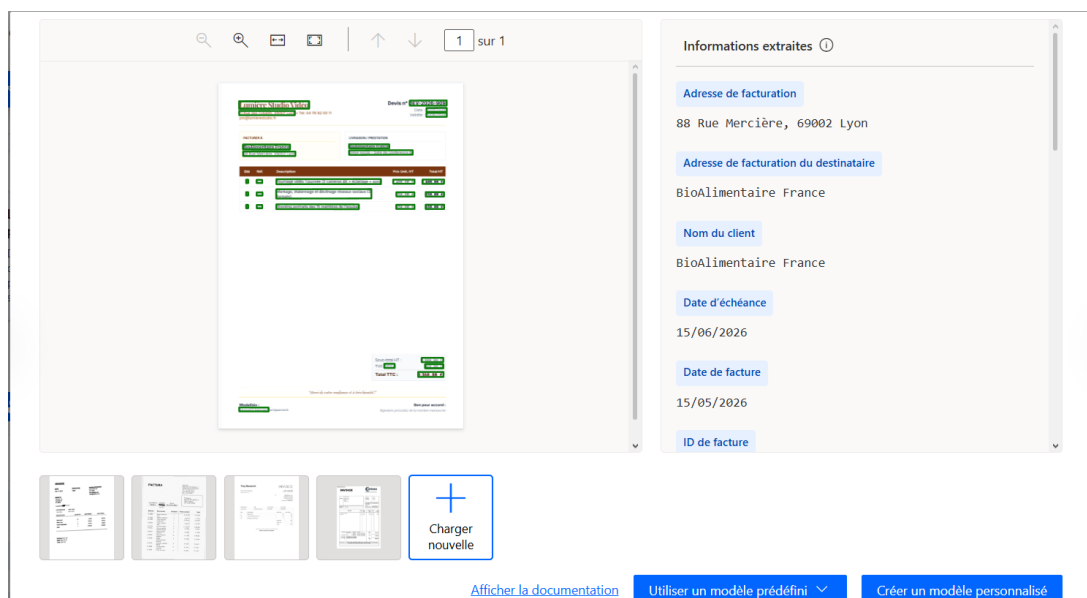


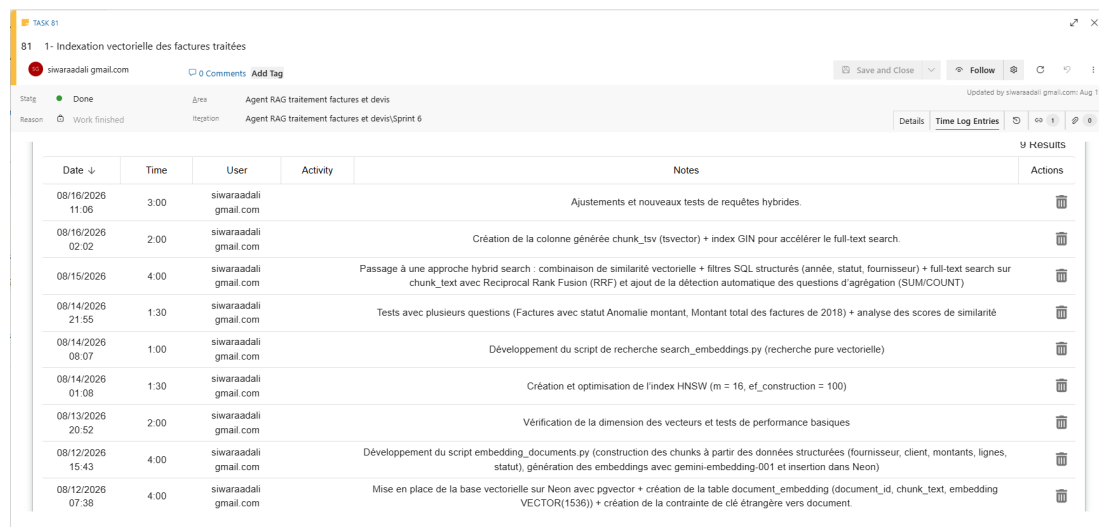
Figure 2.5: Extraction obtenue sur un second devis, distinct du premier, confirmant la cohérence du modèle sur plusieurs documents.

configurations distinctes. Les libellés restituent toutefois un vocabulaire propre à la facturation, comme « *Date de facture* » ou « *Total de la facture* », même lorsqu'ils sont appliqués à un devis. Cette particularité a été prise en compte lors du mappage des champs vers le schéma de données interne présenté au chapitre 3.

Cette approche a permis de conserver une solution compréhensible par les utilisateurs métier de Dazzlifai, tout en maintenant une architecture suffisamment ouverte pour accueillir, dans les sprints suivants, le RAG, le reporting et les traitements Python développés en dehors de l'écosystème Power Platform.

2.5 | Organisation du travail et suivi

Le projet a été mené de façon itérative et découpé en sprints suivis dans Azure DevOps sous la forme de *Backlog Items* et de *Tasks*, chaque élément étant associé à des critères d'acceptation explicites. Chaque tâche a fait l'objet d'un suivi détaillé du temps consacré (*Time Log Entries*). Cette organisation a permis de mesurer précisément l'effort investi dans chaque composante du projet — extraction, intégration Dataverse et développement de l'agent RAG — plutôt que d'en estimer la répartition a posteriori.



The screenshot shows a task log in Azure DevOps for a task titled "1- Indexation vectorielle des factures traitées". The log is organized into a table with columns for Date, Time, User, Activity, and Notes. The entries are sorted by date in descending order. The user for all entries is "siwaraadali@gmail.com". The activities include various development and testing tasks related to vector search, such as creating indexes, testing hybrid search, and optimizing the HNSW index.

Date ↓	Time	User	Activity	Notes
08/16/2026 11:06	3:00	siwaraadali@gmail.com	Ajustements et nouveaux tests de requêtes hybrides.	
08/16/2026 02:02	2:00	siwaraadali@gmail.com	Création de la colonne générée chunk_tsv (tsvector) + index GIN pour accélérer le full-text search.	
08/15/2026 4:00	4:00	siwaraadali@gmail.com	Passage à une approche hybrid search : combinaison de similarité vectorielle + filtres SQL structurés (année, statut, fournisseur) + full-text search sur chunk_text avec Reciprocal Rank Fusion (RRF) et ajout de la détection automatique des questions d'agrégation (SUM/COUNT)	
08/14/2026 21:55	1:30	siwaraadali@gmail.com	Tests avec plusieurs questions (Factures avec statut Anomalie montant, Montant total des factures de 2018) + analyse des scores de similarité	
08/14/2026 08:07	1:00	siwaraadali@gmail.com	Développement du script de recherche search_embeddings.py (recherche pure vectorielle)	
08/14/2026 01:08	1:30	siwaraadali@gmail.com	Création et optimisation de l'index HNSW (m = 16, ef_construction = 100)	
08/13/2026 20:52	2:00	siwaraadali@gmail.com	Vérification de la dimension des vecteurs et tests de performance basiques	
08/12/2026 15:43	4:00	siwaraadali@gmail.com	Développement du script embedding_documents.py (construction des chunks à partir des données structurées (fournisseur, client, montants, lignes, statut), génération des embeddings avec gemini-embedding-001 et insertion dans Neon)	
08/12/2026 07:38	4:00	siwaraadali@gmail.com	Mise en place de la base vectorielle sur Neon avec pgvector + création de la table document_embedding (document_id, chunk_text, embedding VECTOR(1536)) + création de la contrainte de clé étrangère vers document	

Figure 2.6: Exemple de suivi du temps consacré à chaque tâche, consigné dans Azure DevOps.

Cette organisation en sprints courts, avec un incrément livrable à chaque fin de sprint, structure la présentation du chapitre 3 et des suivants : la réalisation y est présentée dans le même ordre chronologique que celui suivi durant le stage.

Chapter 3

Réalisation du pipeline IDP

3.1 | Vue d'ensemble des sprints 1 à 6

Les six premiers sprints ont été consacrés à la construction du pipeline IDP. L'objectif était de passer d'un ensemble de factures et devis stockés dans un dossier dédié à une base de données exploitable par des recherches, des validations et des rapports. Chaque sprint a permis d'ajouter une nouvelle fonctionnalité tout en vérifiant la stabilité des composants déjà développés.

La démarche de réalisation est résumée dans le tableau suivant :

Table 3.1: Principales étapes de réalisation du pipeline IDP

Étape	Objectif
Préparation	Récupérer les documents, identifier leur format et les classer avant leur traitement.
Extraction	Extraire les informations essentielles et les normaliser selon une structure commune.
Contrôle	Vérifier la cohérence des données et utiliser les scores de confiance pour repérer les documents nécessitant une validation humaine.

Chaque étape a été testée indépendamment, puis vérifiée à partir des données réellement enregistrées dans la base. Cette organisation a permis d'obtenir une chaîne de traitement complète et traçable, capable de transformer des documents hétérogènes en informations structurées exploitables par l'agent RAG et les modules de reporting. Elle facilite également la détection rapide des erreurs et la maintenance de chaque composante. L'architecture modulaire permet de modifier une étape sans perturber l'ensemble du pipeline. Enfin, cette conception favorise l'évolution future de la solution vers un volume de documents plus important.

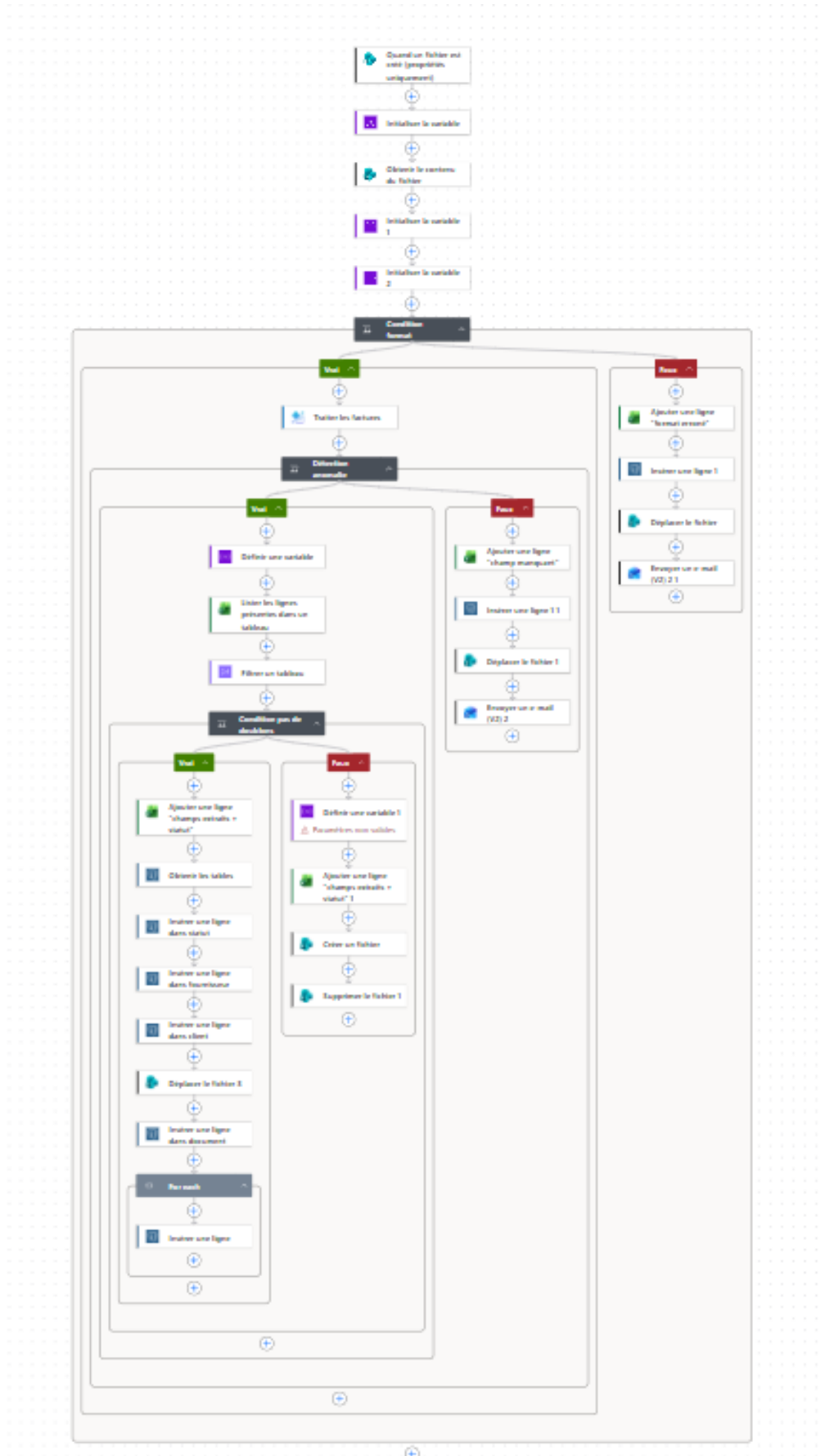


Figure 3.1: Vue d'ensemble du flux Power Automate d'extraction, après intégration complète à la base Postgres : détection du format, extraction, contrôle des doublons, insertion et gestion des anomalies.

Le jeu documentaire de test comprend 1306 fichiers de factures, aux formats variés (PDF, images scannées, photographies), reflétant la diversité réelle des documents reçus par une entreprise : qualité de numérisation inégale, mises en page hétérogènes selon les fournisseurs, et documents parfois incomplets ou mal orientés. Cette diversité, plutôt qu'un jeu de données propre et homogène, a été choisie délibérément : elle seule permet de mettre à l'épreuve la robustesse réelle du pipeline, notamment ses mécanismes de score de confiance et de détection d'anomalies présentés ci-dessous. *[Répartition exacte par format et par type de document à préciser.]*

3.2 | Extraction et validation

La première étape a consisté à extraire les champs essentiels : référence du document, fournisseur, client, date, montant total, lignes de document et statut. Les extractions ont été accompagnées de scores de confiance, notamment `confiance_montant` et `confiance_fournisseur`. Ces valeurs permettent de distinguer un document traité avec assurance d'un document qui doit être revu, plutôt que de traiter chaque extraction avec le même niveau de confiance implicite.

Des contrôles d'anomalies ont été ajoutés pour détecter les cas suspects. Un montant total incohérent avec la somme des lignes de détail, une date d'échéance antérieure à la date d'émission, ou une date absente ne doivent pas être acceptés silencieusement : ces cas sont marqués avec un statut d'anomalie explicite (*Anomalie montant*, *Anomalie date*, ou une combinaison des deux) plutôt que d'être insérés comme des documents ordinaires. Le pipeline enregistre systématiquement ces problèmes dans la table `journal_erreurs` afin de conserver une trace exploitable, y compris pour les cas où l'extraction échoue complètement plutôt que de simplement produire un résultat incohérent.

Cette logique de statut a une conséquence directe sur l'exploitation de la base en aval : un document en anomalie reste consultable et peut être corrigé manuellement, mais il est distinguable dans toutes les requêtes ultérieures — qu'il s'agisse des rapports générés par l'agent (chapitre 5) ou d'un contrôle qualité mené directement en base.

3.2.1 Alerte automatique sur anomalie critique

Certaines anomalies sont jugées suffisamment critiques pour ne pas se limiter à un enregistrement passif en base : c'est le cas lorsqu'un champ essentiel à l'exploitation du document — le montant total ou le nom du fournisseur — revient vide après extraction, ou lorsque le service d'extraction répond avec un code de statut différent de 200. Dans ce cas, le flux Power Automate déclenche l'envoi automatique d'un courriel d'alerte, en plus de l'écriture dans `journal_erreurs`.

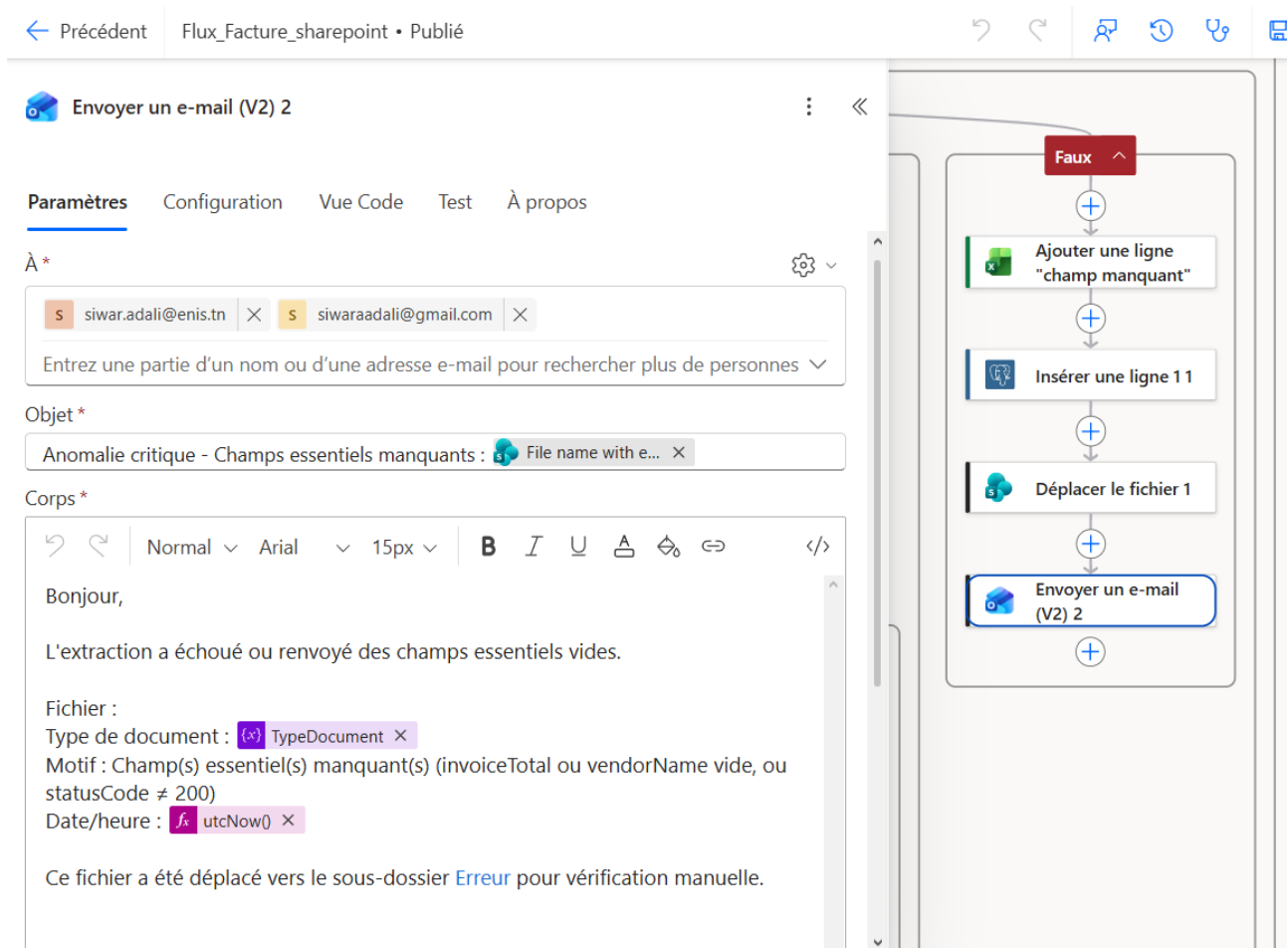


Figure 3.2: Configuration de l'action d'envoi de courriel déclenchée sur anomalie critique, avec le type de document, le motif précis et l'horodatage du traitement.

Le message précise le type de document concerné, le motif exact de l'anomalie (champ essentiel vide ou code de statut anormal) et la date et l'heure du traitement, et indique que le fichier source a été déplacé vers un sous-dossier *Erreur* en attente de vérification manuelle. Cette alerte immédiate distingue les anomalies bloquantes, qui empêchent toute exploitation du document tant qu'elles ne sont pas corrigées, des anomalies non bloquantes (montant ou date incohérents) qui restent consultables en base avec leur statut d'anomalie, sans notification systématique.

3.2.2 Cohérence de l'extraction sur des doublons

Un test complémentaire a porté sur le comportement du pipeline face à des doublons de fichiers. Lorsqu'un même document est déposé plusieurs fois dans le dossier source synchronisé par OneDrive, celui-ci renomme automatiquement les copies suivantes avec un préfixe (*copy_1_*, *copy_2_*, ...) afin d'éviter tout conflit de nom ; chaque copie est ensuite traitée par le pipeline

comme un fichier à part entière.

fichier	typeDocument	invoiceId	vendorName	vendorAddress	customerName	customerAddress
Template12_Instance143.jpg	Facture	5274-277	Richardson-Davis		to:Emily Barnett	Address:6479 Smith Causeway East Camerontown, AS 38
Devis01.pdf	Devis	123	TEMPO Gestion de Patrimoi	123 Anywhere St., Any City	Thomas Garcia	123 Anywhere St., Any City
copy_1_Devis01.pdf	Devis	123	TEMPO Gestion de Patrimoi	123 Anywhere St., Any City	Thomas Garcia	123 Anywhere St., Any City
copy_2_Devis01.pdf	Devis	123	TEMPO Gestion de Patrimoi	123 Anywhere St., Any City	Thomas Garcia	123 Anywhere St., Any City
batch1-0511.pdf	Facture	17014325	Valdez, Clark and Fischer	4108 Marissa Ports New Andreadborough, MN 20857	Hancock-Shields	397 Mary Skyway Suite 261 Rochaburgh, SC 53742
Template16_Instance61.jpg	Facture	INV80586571	DSD LAD	Address:696 Thompson Forest Ashleybury, AR 73994 US	Mr. Daniel Mueller	35147 Lisa Vista Apt. 326 Doughertymouth, TN 55985 US

Figure 3.3: Extraction d'un même devis déposé en trois exemplaires (Devis01.pdf, copy_1_Devis01.pdf, copy_2_Devis01.pdf) : les champs extraits restent identiques sur les trois copies.

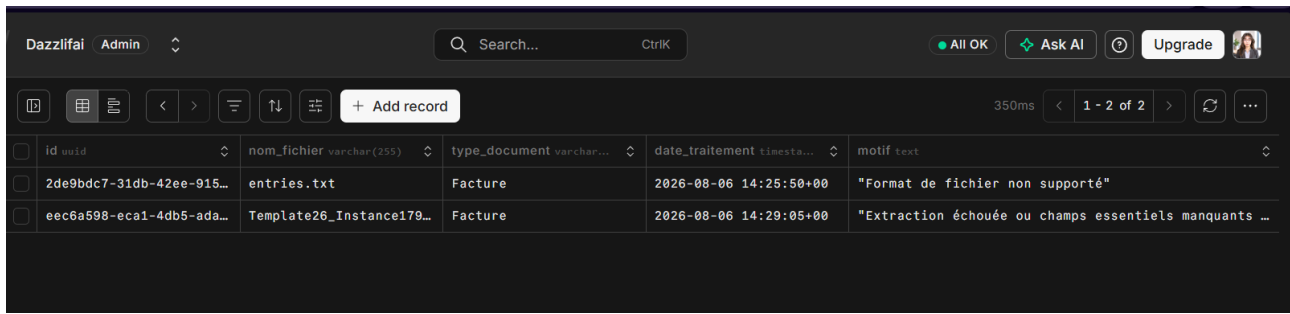
Le test réalisé sur un devis déposé en trois exemplaires confirme que l'extraction reste rigoureusement identique sur les trois copies — même fournisseur, même client, mêmes montants — quel que soit le nom de fichier attribué par le système de synchronisation. Ce test valide la cohérence et le déterminisme de l'étape d'extraction : le résultat dépend uniquement du contenu du document, jamais de son nom de fichier. Il ne constitue en revanche pas, en l'état, un mécanisme de rejet des doublons : chaque copie continue d'être insérée comme un document distinct en base, ce qui reste identifié comme une limite à traiter (voir chapitre 8).

3.2.3 Vérification croisée du journal d'erreurs

À l'image de la démarche de vérification croisée employée plus tard pour valider le moteur de recherche (section 3.6), les erreurs consignées par le pipeline ont été confrontées à deux sources indépendantes : un export Excel généré en parallèle du traitement, et le contenu réel de la table `journal_erreurs` dans Neon.

fichier	typeDocument	dateTraitement	motif
entries.txt	Facture	2026-08-06T14:25:48.0146697Z	Format de fichier non supporté
Template26_Instance179.jpg	Facture	2026-08-06T14:29:03.0174735Z	Extraction échouée ou champs essentiels manquants

Figure 3.4: Export Excel des erreurs rencontrées pendant le traitement, avec le fichier concerné, le type de document supposé, l'horodatage et le motif exact.



id	nom_fichier	type_document	date_traitement	motif
2de9bdc7-31db-42ee-915...	entries.txt	Facture	2026-08-06 14:25:50+00	"Format de fichier non supporté"
eec6a598-eca1-4db5-ada...	Template26_Instance179...	Facture	2026-08-06 14:29:05+00	"Extraction échouée ou champs essentiels manquants ..."

Figure 3.5: Les mêmes erreurs, retrouvées à l'identique dans la table `journal_erreurs` de Neon.

Les deux erreurs observées — un format non supporté et des champs essentiels manquants — apparaissent à l'identique dans les deux sources, avec le même horodatage.

3.3 | Modèle de données Postgres

Les données extraites sont stockées dans PostgreSQL sur Neon. Le modèle comprend les tables suivantes :

- `document` : données générales des factures et devis ;
- `fournisseur` et `client` : informations sur les parties concernées ;
- `statut` : suivi du traitement et du paiement ;
- `ligne_document` : détails des lignes ;
- `journal_erreurs` : historique des anomalies.

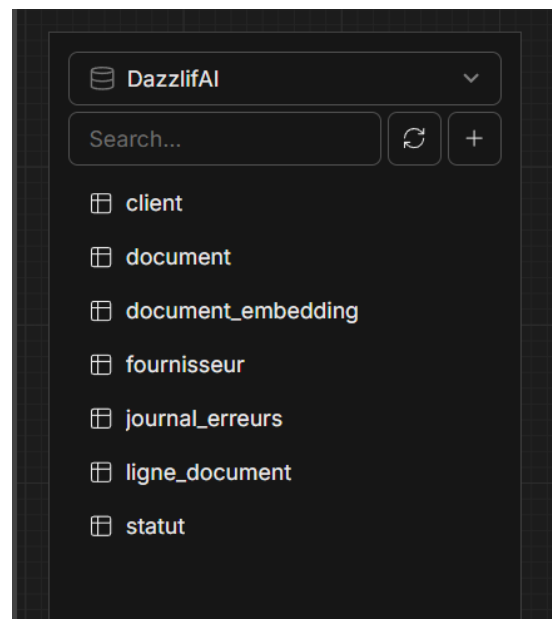


Figure 3.6: Organisation des principales tables de la base `DazzlifAI` dans Neon. La table `document_embedding` est ajoutée lors de l'étape suivante, présentée à la section 3.4.

Ce schéma a été pensé pour séparer les données métier, les informations de suivi et les erreurs. Cette séparation facilite les requêtes analytiques et les contrôles de qualité : il devient possible, par exemple, de compter les documents en anomalie par fournisseur sans avoir à filtrer un champ de statut mélangé aux données métier elles-mêmes.

Tables

🔍 production

DazzlilAI

Schema

public

Search...

client

document

document_embedding

fournisseur

journal_erreurs

ligne_document

statut

v_document_synthese

Figure 3.7: Deux clients réels insérés en base, avec leur document associé consultable directement depuis la table `client` grâce à la relation par clé étrangère.

Cette figure illustre concrètement l'intérêt de la normalisation retenue : consulter le ou les documents rattachés à un client donné ne nécessite pas de requête complexe, la relation entre `client` et `document` étant portée nativement par le schéma plutôt que reconstruite a posteriori.

À l'échelle de plusieurs centaines de documents, ce choix de normalisation a également des conséquences pratiques sur les performances : les jointures entre `document` et `fournisseur` ou `client` restent rapides grâce aux clés étrangères indexées, ce qui s'est avéré important une fois la recherche hybride et les rapports agrégés ajoutés par-dessus ce même schéma (sections 3.5 et 3.6).

3.4 | Génération des embeddings

Un second niveau de traitement a été ajouté avec le script `embedding_documents.py`. Pour chaque document non encore indexé, un texte structuré (*chunk*) est reconstruit à partir des champs extraits — type de document, fournisseur, client, dates, montants, lignes de détail — puis transmis au modèle `gemini-embedding-001` de l'API Gemini, qui retourne un vecteur numérique de 1536 dimensions.

The screenshot shows the DazzlifAI interface. On the left, a sidebar displays the 'production' database schema with tables like 'client', 'document', 'document_embedding', 'fournisseur', 'journal_erreurs', 'ligne_document', 'statut', and 'v_document_synthese'. The main area shows a table with the following columns: document_id, chunk_text, embedding_vector, created_at, and chunk_tsv. The table contains 13 rows of data, each representing a document chunk with its unique ID, text, 1536-dimensional embedding vector, creation timestamp, and a TSV representation of the chunk.

document_id	chunk_text	embedding_vector(1536)	created_at timestamp	chunk_tsv tsvector
004bd676-b2c4-4bdd-960...	Document : Facture Num...	[0.0068085333,0.001163...	2026-08-11 13:28:57.62...	'-02':40 '-06':34
00545f26-7d12-4b2c-83c...	Document : Facture Num...	[0.019778408,-0.013550...	2026-08-12 09:15:53.95...	'-06':46 '-08':47
0147fda0-431e-4ef6-aba...	Document : Facture Num...	[0.017429998,0.0020470...	2026-08-11 13:35:52.17...	'-03':49,50 '-04':
01b6a995-4fc5-4a96-a99...	Document : Facture Num...	[0.014278499,-0.013147...	2026-08-11 13:22:36.83...	'-08':38 '-087':7
01eb08cf-014c-41a6-b4a...	Document : Facture Num...	[-0.0049360967,0.00416...	2026-08-11 13:38:02.12...	'-07':43 '-08':44
01ed0b5f-a812-4958-b4d...	Document : Facture Num...	[0.01799647,0.00669827...	2026-08-12 09:17:43.30...	'-06':43 '-24':44
0211501e-f0f2-4ae6-9b0...	Document : Facture Num...	[0.006140473,-0.006739...	2026-08-12 09:18:31.73...	'-03':47 '-06':46,
025639c4-38fe-4435-8d8...	Document : Facture Num...	[0.021628847,0.0024377...	2026-08-11 13:40:30.28...	'-03':40,46 '-04':
02c10962-10e6-4b35-93e...	Document : Facture Num...	[0.0009416439,-0.01057...	2026-08-11 13:40:32.75...	'-07':42 '-09':35
02ce4bfb-a0df-4590-999...	Document : Facture Num...	[0.02648866,-0.0172292...	2026-08-11 13:32:16.11...	'-950':7 '0':49,7
035006e0-30ac-465b-bb5...	Document : Facture Num...	[0.001790681,0.0016891...	2026-08-11 13:29:01.06...	'-03':41 '-09':48
03f36721-2233-413f-81c...	Document : Facture Num...	[0.0076300753,0.003723...	2026-08-11 13:40:02.68...	'-05':46 '-07':39
04316b0a-eb82-4102-b9b...	Document : Facture Num...	[0.0021607047,-0.01410...	2026-08-11 13:23:22.49...	'-11':36 '-30':37
0496b889-fc98-4e18-bfa...	Document : Facture Num...	[0.007925818,-0.014513...	2026-08-11 13:22:39.15...	'-12':36 '-25':37
0499c3ab-eb9b-405b-8ae...	Document : Facture Num...	[0.015403541,-0.006866...	2026-08-12 09:21:41.08...	'-04':45 '-09':37
050fad0b-9b5a-44aa-bbc...	Document : Facture Num...	[0.01760997,0.00186717...	2026-08-12 09:18:39.01...	'-01':47 '-09':40

Figure 3.8: Construction du texte structuré d'un document et génération de son embedding via l'API Gemini.

Chaque vecteur est ensuite stocké dans la table `document_embedding`, liée à `document` par `document_id`, avec le type `VECTOR(1536)` fourni par l'extension `pgvector`. L'insertion utilise une clause `ON CONFLICT DO UPDATE`, ce qui rend le script rejouable sans risque de doublon : un document déjà indexé et inchangé est simplement ignoré au passage suivant.

id uuid	type_document varchar...	numero_document varchar...	date_emission date	date_echeance_ou_va
dfd7b192-4d13-4e8a-b6f...	Facture	INV38268425	2000-04-03	2009-06-15
82ac27b6-9652-4712-a3f...	Facture	0890-253	NULL	NULL
15779d3e-04e1-4fbb-974...	Facture	5Y2M8d-379	2004-05-15	NULL
b450a894-a02b-46c5-a62...	Facture	1Y2M4d-517	2000-11-14	1998-04-13
2b240ed2-0fb4-4a80-900...	Facture	INV/39-04/404	2018-10-20	NULL
ac2cd9cc-cf9c-4eaa-a3e...	Facture	6893-832	2013-12-09	NULL
4ba47b64-45b9-4307-abd...	Facture	INV88325575	2016-06-06	2015-04-17
1ce7b891-08ec-453c-bad...	Facture	EMPTY_STRING	2009-03-01	2018-03-15
87716f8e-59e5-45d3-97c...	Facture	INV03010608	2011-07-07	1993-07-23
9e47cfa4-990a-4282-b97...	Facture	INV98921338	2019-09-30	2015-11-14
f95f8c3a-e495-4a5d-963...	Facture	1Y9M3d-272	2001-01-01	2005-05-02
0b64c21b-e0e0-4947-aa7...	Facture	8544-550	1993-02-25	NULL
b4d68fa8-5090-48c9-b5a...	Facture	7Y9M9d-923	2001-03-11	NULL
e28073ca-b1be-405c-b33...	Facture	EMPTY_STRING	2013-07-27	NULL
1c7c4025-295e-48cc-acd...	Facture	INV/60-81/529	2019-06-09	NULL
7170072e-9f67-4083-b5b...	Facture	INV42891180	2015-01-11	2010-01-20
0cafd3c2-1372-4d74-8a6...	Facture	INV/45-67/398	1994-10-20	NULL
57a3ed48-ca85-4249-965...	Facture	8V9M8d-868	2012-09-14	NULL

Figure 3.9: Vérification finale affichée par le script, listant chaque document indexé avec la dimension de son vecteur et sa date de création.

Une vérification complémentaire a été effectuée directement depuis la console SQL de Neon, afin de confirmer que l'ensemble des vecteurs stockés respecte bien la dimension attendue avant de passer à l'étape de recherche. La requête de contrôle affiche l'identifiant de chaque document ainsi que le nombre de dimensions de son embedding.

```

1 SELECT
2   COUNT(*) AS total_embeddings,
3   COUNT(embedding) AS embeddings_non_null,
4   MIN(vector_dims(embedding)) AS min_dimensions,
5   MAX(vector_dims(embedding)) AS max_dimensions
6 FROM public.document_embedding;

```

#	total_embeddings	embeddings_non_null	min_dimensions	max_dimensions
1	1306	1306	1536	1536

Figure 3.10: Contrôle, depuis la console Neon, que chaque embedding stocké possède bien 1536 dimensions.

À l'échelle du jeu documentaire complet, l'indexation de l'ensemble des documents n'a pas pu s'effectuer en une seule exécution ininterrompue : c'est précisément ce traitement de volume qui a révélé la fragilité de connexion à la base décrite au chapitre 6, la connexion Neon se coupant après un nombre significatif de documents traités. Le script d'indexation a donc été rendu idempotent par construction (grâce à la clause `ON CONFLICT DO UPDATE` mentionnée plus haut), ce qui a permis de le relancer autant de fois que nécessaire jusqu'à couverture complète du jeu de test, sans jamais dupliquer un embedding déjà généré.

3.5 | Moteur de recherche hybride

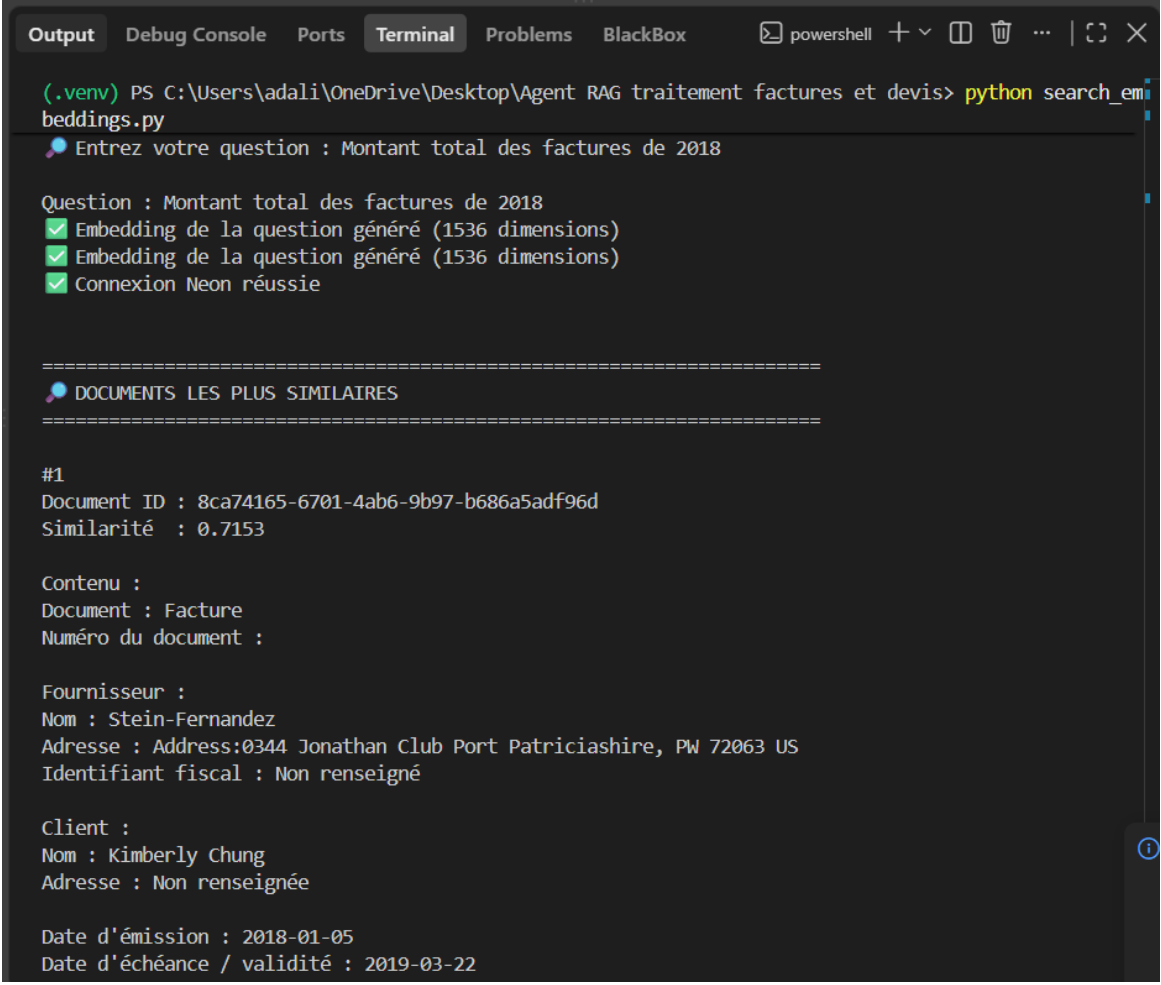
Le moteur de recherche a d'abord été implémenté dans `search_embeddings.py`, puis refactorisé dans `rag_search.py`. Il combine, comme présenté sur le plan théorique au chapitre 1, une recherche vectorielle, une recherche full-text PostgreSQL et des filtres structurés comme l'année, le statut, le fournisseur ou le statut de paiement, avant de fusionner les deux classements par la méthode RRF.



Figure 3.11: Exemples de questions en langage naturel utilisées pour tester les capacités de recherche, de filtrage et d'agrégation du moteur hybride.

Concrètement, chaque question posée est d'abord analysée pour en extraire d'éventuels filtres structurés (une année, un nom de fournisseur, un statut), transformée en vecteur, puis exécutée en parallèle comme requête vectorielle et comme requête plein texte. Ce découpage permet, par

exemple, à une question combinant un critère précis (un fournisseur nommé explicitement) et une formulation ouverte (« *des anomalies* ») de bénéficier des deux mécanismes de recherche simultanément, plutôt que de forcer l'un des deux à porter seul toute la charge de la pertinence. Les deux exemples suivants illustrent ce comportement sur des questions réelles posées pendant les tests du script.



```
(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis> python search_embeddings.py
Entrez votre question : Montant total des factures de 2018

Question : Montant total des factures de 2018
[✓] Embedding de la question généré (1536 dimensions)
[✓] Embedding de la question généré (1536 dimensions)
[✓] Connexion Neon réussie

=====
DOCUMENTS LES PLUS SIMILAIRES
=====

#1
Document ID : 8ca74165-6701-4ab6-9b97-b686a5adf96d
Similarité : 0.7153

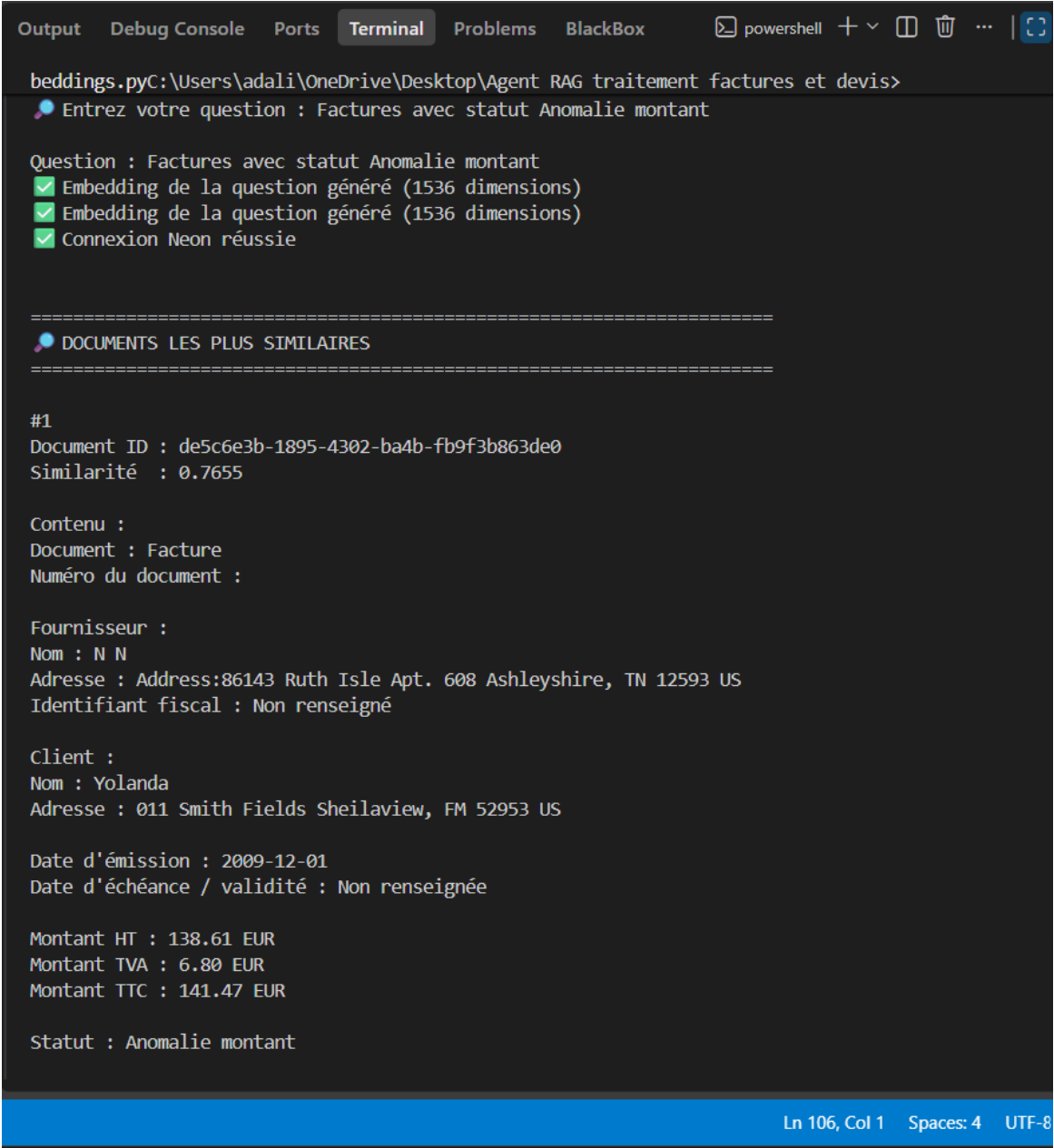
Contenu :
Document : Facture
Numéro du document :

Fournisseur :
Nom : Stein-Fernandez
Adresse : Address:0344 Jonathan Club Port Patriciashire, PW 72063 US
Identifiant fiscal : Non renseigné

Client :
Nom : Kimberly Chung
Adresse : Non renseignée

Date d'émission : 2018-01-05
Date d'échéance / validité : 2019-03-22
```

Figure 3.12: Exécution du moteur de recherche hybride sur une question en langage naturel, avec les scores vectoriel, textuel et RRF affichés pour chaque document retrouvé.



```
beddings.pyC:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis>
Entrez votre question : Factures avec statut Anomalie montant

Question : Factures avec statut Anomalie montant
✓ Embedding de la question généré (1536 dimensions)
✓ Embedding de la question généré (1536 dimensions)
✓ Connexion Neon réussie

=====
DOCUMENTS LES PLUS SIMILAIRES
=====

#1
Document ID : de5c6e3b-1895-4302-ba4b-fb9f3b863de0
Similarité : 0.7655

Contenu :
Document : Facture
Numéro du document :

Fournisseur :
Nom : N N
Adresse : Address:86143 Ruth Isle Apt. 608 Ashleyshire, TN 12593 US
Identifiant fiscal : Non renseigné

Client :
Nom : Yolanda
Adresse : 011 Smith Fields Sheilaview, FM 52953 US

Date d'émission : 2009-12-01
Date d'échéance / validité : Non renseignée

Montant HT : 138.61 EUR
Montant TVA : 6.80 EUR
Montant TTC : 141.47 EUR

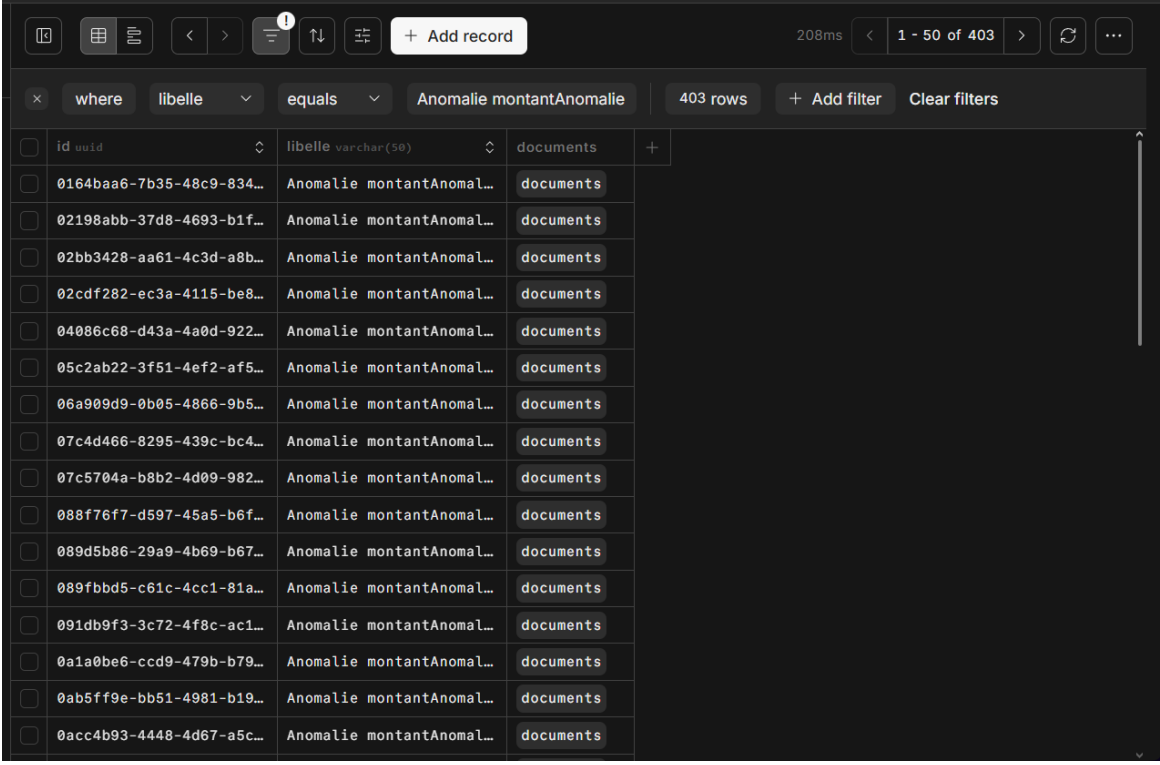
Statut : Anomalie montant

Ln 106, Col 1 Spaces: 4 UTF-8
```

Figure 3.13: Second exemple d'exécution, illustrant le classement obtenu sur une question différente.

3.6 | Vérification et validation du classement

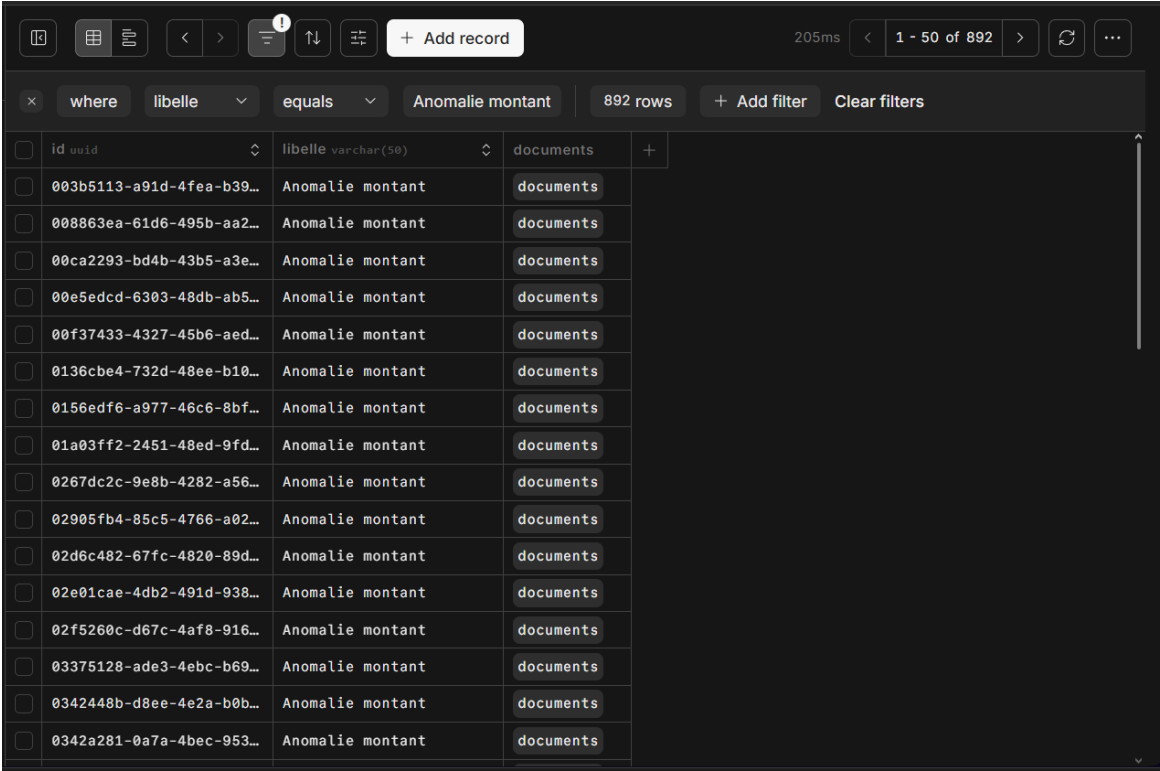
Avant d'intégrer ce moteur dans l'agent final, chaque type de requête a été validé indépendamment en confrontant les résultats retournés par le script aux données réellement présentes dans Neon, afin de s'assurer que le classement produit correspondait bien à la réalité de la base plutôt qu'à un artefact de calcul. Cette vérification a porté sur la pertinence des documents sélectionnés, l'ordre du classement et l'application correcte des filtres structurés.



The screenshot shows a database interface with a query result table. The table has three columns: `id uuid`, `libelle varchar(50)`, and `documents`. The query is `where libelle equals Anomalie montantAnomalie`. The results show 403 rows, with the first 15 rows displayed. Each row contains a unique UUID, the text 'Anomalie montantAnomalie', and a 'documents' link.

id uuid	libelle varchar(50)	documents
0164baa6-7b35-48c9-834...	Anomalie montantAnomalie	documents
02198abb-37d8-4693-b1f...	Anomalie montantAnomalie	documents
02bb3428-aa61-4c3d-a8b...	Anomalie montantAnomalie	documents
02cdf282-ec3a-4115-be8...	Anomalie montantAnomalie	documents
04086c68-d43a-4a0d-922...	Anomalie montantAnomalie	documents
05c2ab22-3f51-4ef2-af5...	Anomalie montantAnomalie	documents
06a909d9-0b05-4866-9b5...	Anomalie montantAnomalie	documents
07c4d466-8295-439c-bc4...	Anomalie montantAnomalie	documents
07c5704a-b8b2-4d09-982...	Anomalie montantAnomalie	documents
088f76f7-d597-45a5-b6f...	Anomalie montantAnomalie	documents
089d5b86-29a9-4b69-b67...	Anomalie montantAnomalie	documents
089fbbd5-c61c-4cc1-81a...	Anomalie montantAnomalie	documents
091db9f3-3c72-4f8c-ac1...	Anomalie montantAnomalie	documents
0a1a0be6-ccd9-479b-b79...	Anomalie montantAnomalie	documents
0ab5ff9e-bb51-4981-b19...	Anomalie montantAnomalie	documents
0acc4b93-4448-4d67-a5c...	Anomalie montantAnomalie	documents

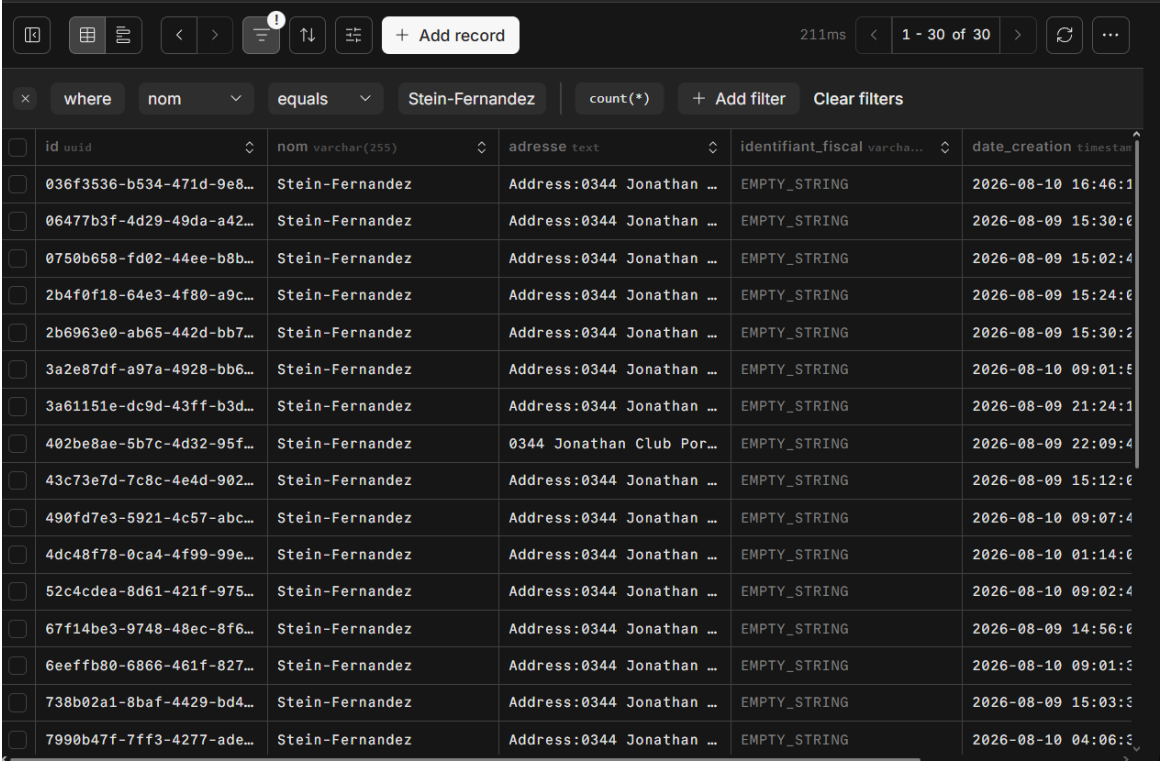
Figure 3.14: Confrontation des documents les mieux classés par la recherche hybride avec une requête directe dans Neon.



The screenshot shows a database interface with a query result table. The table has three columns: `id uuid`, `libelle varchar(50)`, and `documents`. The query is `where libelle equals Anomalie montant`. The results show 892 rows, with the first 15 rows displayed. Each row contains a unique UUID, the text 'Anomalie montant', and a 'documents' link.

id uuid	libelle varchar(50)	documents
003b5113-a91d-4fea-b39...	Anomalie montant	documents
008863ea-61d6-495b-aa2...	Anomalie montant	documents
00ca2293-bd4b-43b5-a3e...	Anomalie montant	documents
00e5edcd-6303-48db-ab5...	Anomalie montant	documents
00f37433-4327-45b6-aed...	Anomalie montant	documents
0136cbe4-732d-48ee-b10...	Anomalie montant	documents
0156edf6-a977-46c6-8bf...	Anomalie montant	documents
01a03ff2-2451-48ed-9fd...	Anomalie montant	documents
0267dc2c-9e8b-4282-a56...	Anomalie montant	documents
02905fb4-85c5-4766-a02...	Anomalie montant	documents
02d6c482-67fc-4820-89d...	Anomalie montant	documents
02e01cae-4db2-491d-938...	Anomalie montant	documents
02f5260c-d67c-4af8-916...	Anomalie montant	documents
03375128-ade3-4ebc-b69...	Anomalie montant	documents
0342448b-d8ee-4e2a-b0b...	Anomalie montant	documents
0342a281-0a7a-4bec-953...	Anomalie montant	documents

Figure 3.15: Second contrôle de cohérence entre les résultats du moteur et le contenu brut de la base.



The screenshot shows a database query interface with a table of records. The table has columns: id uuid, nom varchar(255), adresse text, identifiant_fiscal varchar..., and date_creation timestamp. The filter 'where nom equals Stein-Fernandez' is applied. The table contains 15 records, all with 'Stein-Fernandez' in the 'nom' column. The 'adresse' column contains various addresses, mostly 'Address:0344 Jonathan ...'. The 'date_creation' column shows dates from 2026-08-09 to 2026-08-10.

id uuid	nom varchar(255)	adresse text	identifiant_fiscal varchar...	date_creation timestamp
036f3536-b534-471d-9e8...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 16:46:1
06477b3f-4d29-49da-a42...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:30:0
0750b658-fd02-44ee-b8b...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:02:4
2b4f0f18-64e3-4f80-a9c...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:24:0
2b6963e0-ab65-442d-bb7...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:30:2
3a2e87df-a97a-4928-bb6...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 09:01:5
3a61151e-dc9d-43ff-b3d...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 21:24:1
402be8ae-5b7c-4d32-95f...	Stein-Fernandez	0344 Jonathan Club Por...	EMPTY_STRING	2026-08-09 22:09:4
43c73e7d-7c8c-4e4d-902...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:12:0
490fd7e3-5921-4c57-abc...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 09:07:4
4dc48f78-0ca4-4f99-99e...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 01:14:0
52c4cdea-8d61-421f-975...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 09:02:4
67f14be3-9748-48ec-8f6...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 14:56:0
6eeffb80-6866-461f-827...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 09:01:3
738b02a1-8baf-4429-bd4...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-09 15:03:3
7990b47f-7ff3-4277-ade...	Stein-Fernandez	Address:0344 Jonathan ...	EMPTY_STRING	2026-08-10 04:06:3

Figure 3.16: Vérification spécifique du filtre structuré sur le nom du fournisseur, croisée avec Neon.

La branche d'agrégation du script est utilisée lorsque la question porte sur une valeur calculée, comme un total ou un dénombrement, plutôt que sur la recherche d'un document similaire. Elle transforme alors la demande de l'utilisateur en une opération SQL adaptée et applique, si nécessaire, les filtres détectés dans la question. Trois cas représentatifs ont été validés au cours du développement :

- **Calcul d'un montant total** : le moteur additionne les montants des documents correspondant aux critères de la question et retourne une valeur unique, accompagnée du périmètre pris en compte.
- **Dénombrement des anomalies** : une opération de comptage permet d'identifier le nombre de factures présentant une anomalie de montant, sans dépendre du classement sémantique des documents.
- **Agrégation par fournisseur** : le calcul est combiné à un filtre sur le nom du fournisseur afin de limiter précisément le résultat aux documents concernés.

Pour chaque scénario, le résultat produit par le script a été comparé à une requête exécutée directement dans Neon. Cette vérification a permis de contrôler la valeur calculée, le nombre de documents sélectionnés et la bonne application des filtres.

```

beddings.pyC:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis>
Ligne 1 :
Description : Guy address.
Quantité :
-----

✅ Recherche terminée.
• beddings.pyC:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis> python search_

Entrez votre question : Montant total des factures de 2018

Question : Montant total des factures de 2018

Filtres détectés : {'year': 2018, 'statut': None, 'fournisseur': None, 'is_aggregation': True, 'agg_type': 'sum'}
✅ Embedding généré (1536 dimensions)
✅ Connexion Neon réussie

=====
RÉSULTAT D'AGRÉGATION (SQL + filtres)
=====
Nombre de factures : 51
Total TTC          : 29569.75
Total HT           : 21709.64
Total TVA          : 1143.31

✅ Recherche hybride terminée.
(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis>

```

Ln 221, Col 41 Spaces: 4 UTF-8

Figure 3.17: Validation de la branche d'agrégation SQL sur une question portant sur un montant total.

```

(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis> python search_e
• beddings.py

Entrez votre question : Nombre de factures avec anomalie montant

Question : Nombre de factures avec anomalie montant

Filtres détectés : {'year': None, 'statut': 'Anomalie montant', 'fournisseur': None, 'is_aggregation': True, 'agg_type': 'count'}
✅ Embedding généré (1536 dimensions)
✅ Connexion Neon réussie

=====
RÉSULTAT D'AGRÉGATION (SQL + filtres)
=====
Nombre de factures : 1294

✅ Recherche hybride terminée.
(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis>

```

Ln 221, Col 41 Spaces: 4 UTF-8

Figure 3.18: Validation du dénombrement du nombre de factures en anomalie de montant.

```
(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis> python search_em
beddings.py

Entrez votre question : nombre de Factures du fournisseur Stein-Fernandez

Question : nombre de Factures du fournisseur Stein-Fernandez

Filtres détectés : {'year': None, 'statut': None, 'fournisseur': 'Stein-Fernandez', 'is_aggregat
ion': True, 'agg_type': 'count'}
✓ Embedding généré (1536 dimensions)
✓ Connexion Neon réussie

=====
RÉSULTAT D'AGRÉGATION (SQL + filtres)
=====

Nombre de factures : 30

✓ Recherche hybride terminée.
```

Figure 3.19: Validation de l'agrégation combinée à un filtre nommé sur le fournisseur.

Cette phase de vérification, bien qu'antérieure à l'intégration finale dans l'agent conversationnel présentée au chapitre 5, s'est révélée déterminante : elle a permis de valider indépendamment la fiabilité du moteur de recherche avant de lui superposer la couche de génération par le modèle de langage, et donc de distinguer plus facilement, en cas d'erreur ultérieure, ce qui relevait du moteur de recherche de ce qui relevait de la génération.

3.7 | Bilan du pipeline IDP

Au terme de ces six premiers sprints, le pipeline transforme un document brut en une ligne structurée, validée et indexée sémantiquement, interrogeable à la fois par requête SQL classique et par similarité vectorielle. C'est cette double capacité — données fiables d'un côté, recherche sémantique de l'autre — qui a permis d'aborder les sprints suivants (intégration à l'outil de facturation, puis agent RAG final) sans avoir à revenir sur les fondations posées ici.

Chapter 4

Intégration avec l'outil de facturation

4.1 | Objectif du sprint 7

Le sprint 7 correspondait au Backlog Item 71 et aux tâches 83 et 84. Son objectif était d'intégrer les données extraites avec un outil de facturation accessible via Power Apps. La solution devait permettre une synchronisation bidirectionnelle entre Postgres, Microsoft Dataverse et l'interface utilisateur, sans dupliquer la logique de calcul entre les trois environnements.



Figure 4.1: Architecture globale de synchronisation entre Postgres, Dataverse et Power Apps : Postgres reste la source de vérité, Dataverse en porte une copie opérationnelle, et deux flux Power Automate assurent la synchronisation dans les deux sens.

4.2 | Synchronisation Postgres vers Dataverse

La création de la copie Dataverse ne constitue pas un flux indépendant : elle prolonge directement le flux `Flux_Facture_sharepoint` déjà présenté au chapitre 3 (figure 3.1). Une fois le document inséré dans Postgres et la vérification de doublon effectuée, le même flux enchaîne sur la création de l'entrée Dataverse correspondante, avant de renvoyer l'identifiant généré vers la colonne `id_dataverse` de Postgres.

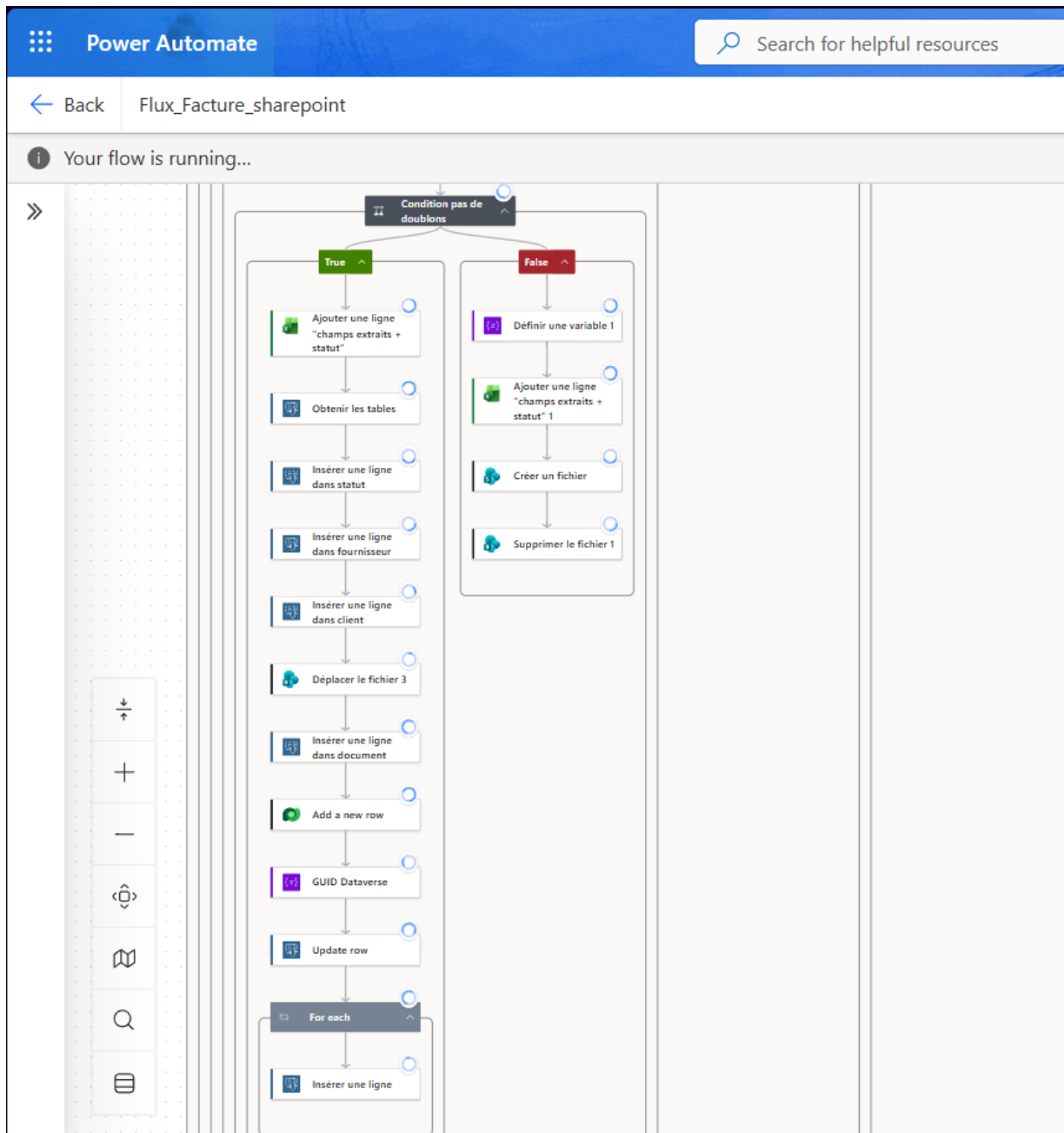
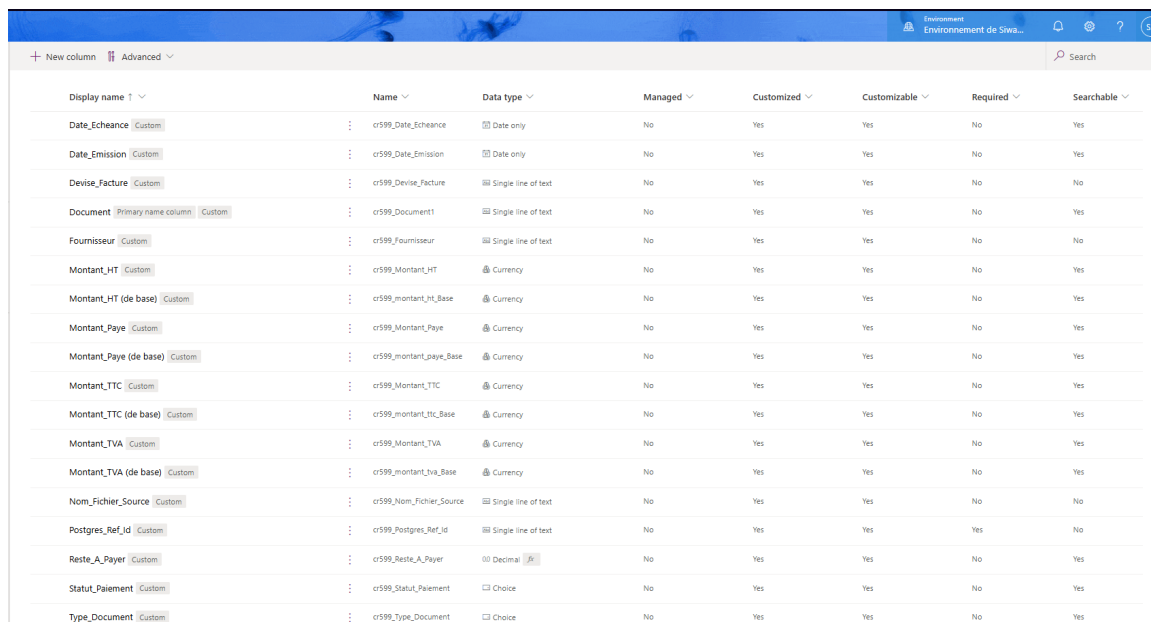


Figure 4.2: Suite du flux `Flux_Facture_sharepoint` : après la branche *Condition pas de doublons*, insertion dans les tables métier, puis création de la ligne Dataverse (*Add a new row*), récupération du GUID et mise à jour de `id_dataverse` côté Postgres.

La modélisation Dataverse a été construite avec des colonnes correspondant terme à terme au schéma Postgres : chaque champ extrait (Montant_HT, Montant_TVA, Montant_TTC, dates, fournisseur) y est dupliqué, avec en complément un champ `Postgres_Ref_Id` marqué comme requis, qui sert de clé de liaison retour vers la ligne d'origine.



Display name	Name	Data type	Managed	Customized	Customizable	Required	Searchable
Date_Echeance	cr599_Date_Echeance	Date only	No	Yes	Yes	No	Yes
Date_Emission	cr599_Date_Emission	Date only	No	Yes	Yes	No	Yes
Devise_Facture	cr599_Devise_Facture	Single line of text	No	Yes	Yes	No	No
Document	cr599_Document1	Single line of text	No	Yes	Yes	No	Yes
Fournisseur	cr599_Fournisseur	Single line of text	No	Yes	Yes	No	No
Montant_HT	cr599_Montant_HT	Currency	No	Yes	Yes	No	Yes
Montant_HT (de base)	cr599_montant_ht_Base	Currency	No	Yes	Yes	No	Yes
Montant_Paye	cr599_Montant_Paye	Currency	No	Yes	Yes	No	Yes
Montant_Paye (de base)	cr599_montant_paye_Base	Currency	No	Yes	Yes	No	Yes
Montant_TTC	cr599_Montant_TTC	Currency	No	Yes	Yes	No	Yes
Montant_TTC (de base)	cr599_montant_ttc_Base	Currency	No	Yes	Yes	No	Yes
Montant_TVA	cr599_Montant_TVA	Currency	No	Yes	Yes	No	Yes
Montant_TVA (de base)	cr599_montant_tva_Base	Currency	No	Yes	Yes	No	Yes
Nom_Fichier_Source	cr599_Nom_Fichier_Source	Single line of text	No	Yes	Yes	No	No
Postgres_Ref_Id	cr599_Postgres_Ref_Id	Single line of text	No	Yes	Yes	Yes	No
Reste_A_Payer	cr599_Reste_A_Payer	Decimal	No	Yes	Yes	No	Yes
Statut_Paiement	cr599_Statut_Paiement	Choice	No	Yes	Yes	No	Yes
Type_Document	cr599_Type_Document	Choice	No	Yes	Yes	No	Yes

Figure 4.3: Colonnes de la table Dataverse Document. Le champ `Reste_A_Payer` est configuré en colonne calculée (icône *fx*), et non en simple valeur saisie, ce qui garantit sa cohérence automatique avec `Montant_TTC` et `Montant_Paye`.

Une fois synchronisée, la table `Documents` de Dataverse reflète l'intégralité des documents traités par le pipeline, avec leur statut de paiement individuel. Le tableau 4.1 résume les informations conservées sur chaque ligne et leur rôle pour l'utilisateur métier.

Table 4.1: Principales informations synchronisées dans Dataverse

Information	Rôle
Référence et fournisseur	Identification et classement du document
Dates d'émission et d'échéance	Suivi des délais de règlement
Montants et statut	Suivi des paiements et du reste à payer
Postgres_Ref_Id	Liaison avec la donnée d'origine

Cette organisation permet aux utilisateurs de consulter rapidement l'état de chaque facture, de repérer les paiements partiels et de suivre les montants restant à régler depuis une interface centralisée.

Montant_...	Montant_...	Devise_Facture	Fournisseur	Nom_Fichier_Source	Postgres_Ref_Id	Montant_...	Reste_A_P...	Statut_Pai...	Propriétaire
0,00	604,07	EUR	Miller Inc	Template32_Instance125.jpg	5c5af871-1ad3-49ae-b56a-351f69...	0,00	604,07	Impayée	Siwar Adali
0,00	575,13	EUR	Miller Inc	Template32_Instance178.jpg	e794912a-62c8-4f4a-8ede-dbe3a...	0,00	575,13	Impayée	Siwar Adali
0,00	207,44	EUR	Kelly-Moss	Template25_Instance40.jpg	abc8cf2c-4766-4f4f-a1eb-cd168...	0,00	207,44	Impayée	Siwar Adali
51,33	591,71	USD	White and Sons	Template7_Instance81.jpg	d23c2950-f289-49c6-8536-bbb3a...	0,00	591,71	Impayée	Siwar Adali
24,44	485,61	USD	Kirk, Murphy and Daniels	Template3_Instance11.jpg	cc4869bd-bd31-4a66-b76f-e0ff41...	0,00	485,61	Impayée	Siwar Adali
0,00	177,80	USD	King, Richards and Roy	Template23_Instance187.jpg	173a1f7f-cc21-4411-a67f-b7bb6e...	0,00	177,80	Impayée	Siwar Adali
87,65	980,47	USD	CALLLENTY	Template48_Instance113.jpg	2ecaca90-7c31-4b6d-a820-e9bbb...	0,00	980,47	Impayée	Siwar Adali
0,00	1 533,97	USD	Navarro, Ford and Bryan	Template9_Instance136.jpg	ea1d2c30-f147-4ca7-a54e-21cfb0...	0,00	1 533,97	Impayée	Siwar Adali
0,00	134,01	USD	Navarro, Ford and Bryan	Template9_Instance96.jpg	db4be353-128c-40db-aad1-317f9...	100,00	34,01	Partiellement ...	Siwar Adali
10,36	307,32	USD	Knight, Huffman and Moore	Template44_Instance71.jpg	3a323dae-13ae-493f-b392-4d206...	0,00	307,32	Impayée	Siwar Adali
37,42	420,97	USD	White and Sons	Template7_Instance186.jpg	b6682290-4fde-40bf-8a4c-3fdea8...	0,00	420,97	Impayée	Siwar Adali
0,00	812,59	USD	DS	Template40_Instance144.jpg	05a67a89-5647-4a75-83d4-36cb2...	0,00	812,59	Impayée	Siwar Adali
10,69	166,71	USD	SIMPLEHY	Template18_Instance136.jpg	f77614af-921e-43d6-92c3-bc943...	0,00	166,71	Impayée	Siwar Adali
16,64	484,02	USD	Phillips, Thomas and Malone	Template6_Instance46.jpg	ea82eee2-6552-4795-9bb4-dcae...	0,00	484,02	Impayée	Siwar Adali
9,26	249,15	EUR	SIMPLEHY	Template18_Instance51.jpg	7a5725ba-111c-428d-b4dc-3cd67...	0,00	249,15	Impayée	Siwar Adali
33,67	191,03	USD	Duran, Morgan and Lopez	Template33_Instance132.jpg	2112ce41-a9e6-4d9b-91e1-ed738...	0,00	191,03	Impayée	Siwar Adali
0,00	464,71	USD	LCO	Template35_Instance32.jpg	3f0e4ba7-0320-4668-997e-3d7b9...	0,00	464,71	Impayée	Siwar Adali
0,00	477,01	USD	Graham, Camillo and Clark	Template8_Instance7.jpg	70b30501-0ba4-4315-86a5-7afa...	0,00	477,01	Impayée	Siwar Adali

Figure 4.4: Table Documents après synchronisation. La ligne *Navarro, Ford and Bryan* illustre un paiement partiel (100,00 réglés sur 134,01, soit 34,01 de reste à payer), avec le statut *Partiellement payée* calculé automatiquement.

4.3 | Formulaire Power Apps

Le formulaire Power Apps, intitulé *Outil de facturation*, centralise la consultation et le suivi des documents synchronisés. Les informations issues de l'extraction sont affichées en lecture seule afin de préserver l'intégrité des données sources. Seul le champ **Montant_Paye** peut être renseigné par l'utilisateur ; sa modification déclenche ensuite la mise à jour du reste à payer et du statut de paiement. Cette séparation permet de sécuriser les données extraites tout en offrant une interaction simple pour le suivi financier.

Les principales fonctionnalités proposées à l'utilisateur sont les suivantes :

- consulter la liste des factures et des devis synchronisés depuis Dataverse ;
- afficher les informations détaillées du document sélectionné ;
- vérifier les montants, les dates, le fournisseur et le statut de paiement ;
- saisir le montant payé sans modifier les données extraites automatiquement ;
- déclencher la synchronisation du paiement vers PostgreSQL après l'enregistrement.

Documents actifs*

Document	Type_Document	Date_Emission	Date_Echeance	Montant_HT	Montant_TVA	Montant_TTC	Devise_Facture
0626-726	Facture	10/07/2016	04/12/2017	952,47 €	58,84 €	976,93 €	EUR
0702-010	Facture	24/02/2017	08/11/2003	454,70 €	20,10 €	455,75 €	EUR
0760-783	Facture	18/05/2008	17/08/1999	280,17 €	12,30 €	289,14 €	EUR
0824-073	Facture	28/09/2012	29/10/2006	0,00 €	12,56 €	362,03 €	USD
0845-822	Facture	29/09/2006		604,01 €	0,00 €	600,38 €	USD
0890-253	Facture			934,20 €	0,00 €	926,29 €	USD
0898-014	Facture	24/12/2010	30/10/2010	709,15 €	0,00 €	733,51 €	USD
0916-074	Facture	06/07/1995	29/10/2010	0,00 €	19,36 €	543,60 €	EUR
0934-124	Facture	14/04/1996		392,92 €	0,00 €	400,38 €	USD
0949-616	Facture	23/07/2008	04/03/2013	301,87 €	27,17 €	316,03 €	USD
0970-395	Facture	26/03/2011		0,00 €	0,00 €	962,03 €	USD
102	Facture	21/02/2022	11/03/2022	744,00 €	59,52 €	853,72 €	CAD

Nb de lignes : 1149

Figure 4.5: Vue en liste de l'outil de facturation, avec 1149 documents actifs affichés — un volume cohérent avec celui observé côté Postgres (figure 4.8), ce qui confirme l'exhaustivité de la synchronisation.

0626-726 - Enregistré

Document

Général Association

Assistant de formulaires

Date_Echeance	04/12/2017
Montant_HT	952,47 €
Montant_TVA	58,84 €
Montant_TTC	976,93 €
Devise_Facture	EUR
Fournisseur	AT
Nom_Fichier_Source	Template43_Instance99.jpg
Postgres_Ref_Id	08b30622-5632-4a16-a83b-38dd3170011e
Montant_Paye	0,00 €
Statut_Paiement	Impayée
Reste_A_Payer	976,93

Figure 4.6: Formulaire de détail du document 0626-726 : champs extraits verrouillés (cadenas), Postgres_Ref_Id requis, Montant_Paye et Statut_Paiement restant les seuls champs modifiables.

4.4 | Synchronisation du paiement

Le deuxième flux Power Automate, *Synchronisation_Paiement*, se déclenche sur toute modification d'une ligne Dataverse et répercute le montant payé vers Postgres, avant de mettre à jour le statut de paiement selon trois cas : intégralement payé, partiellement payé, ou impayé.

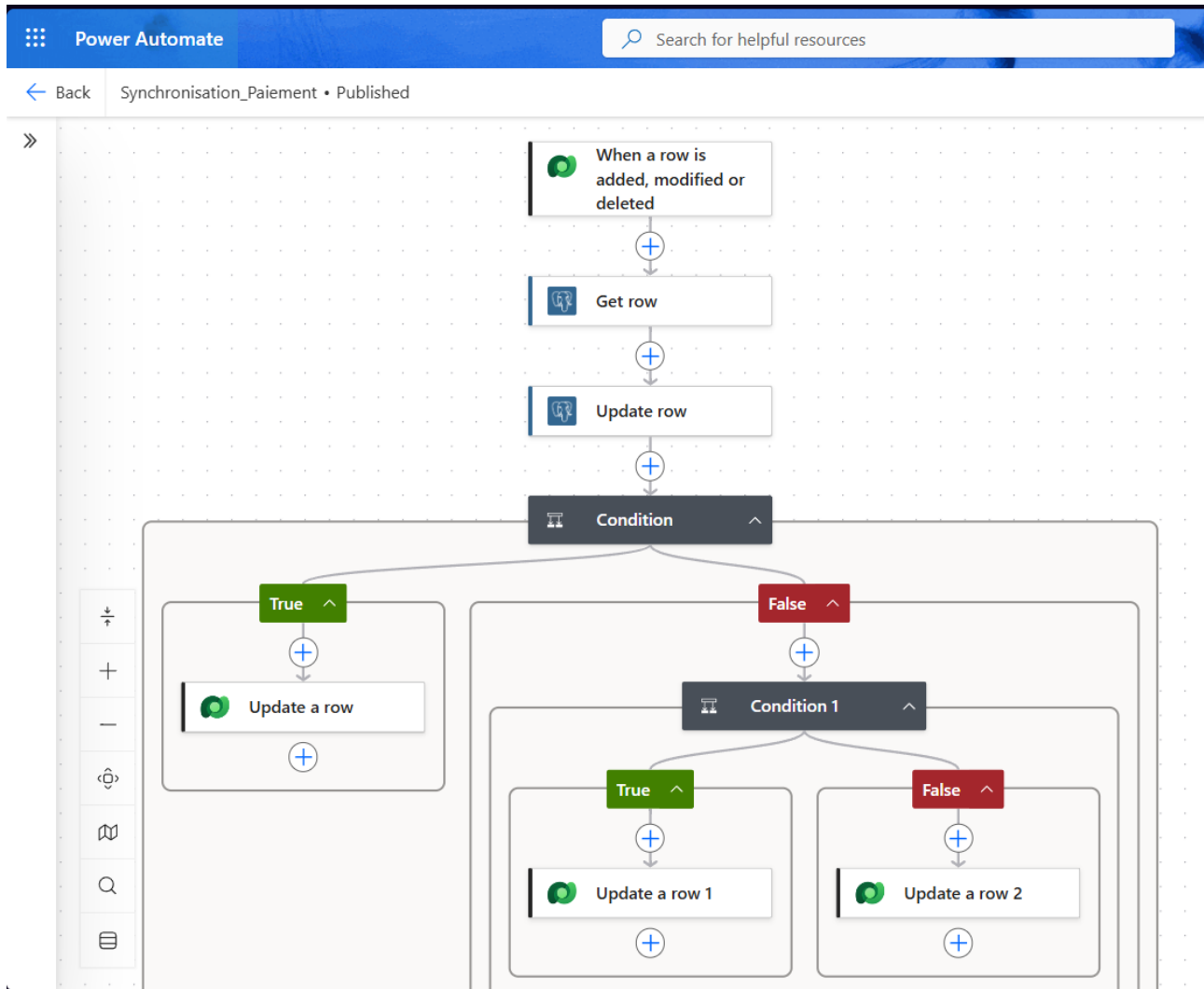
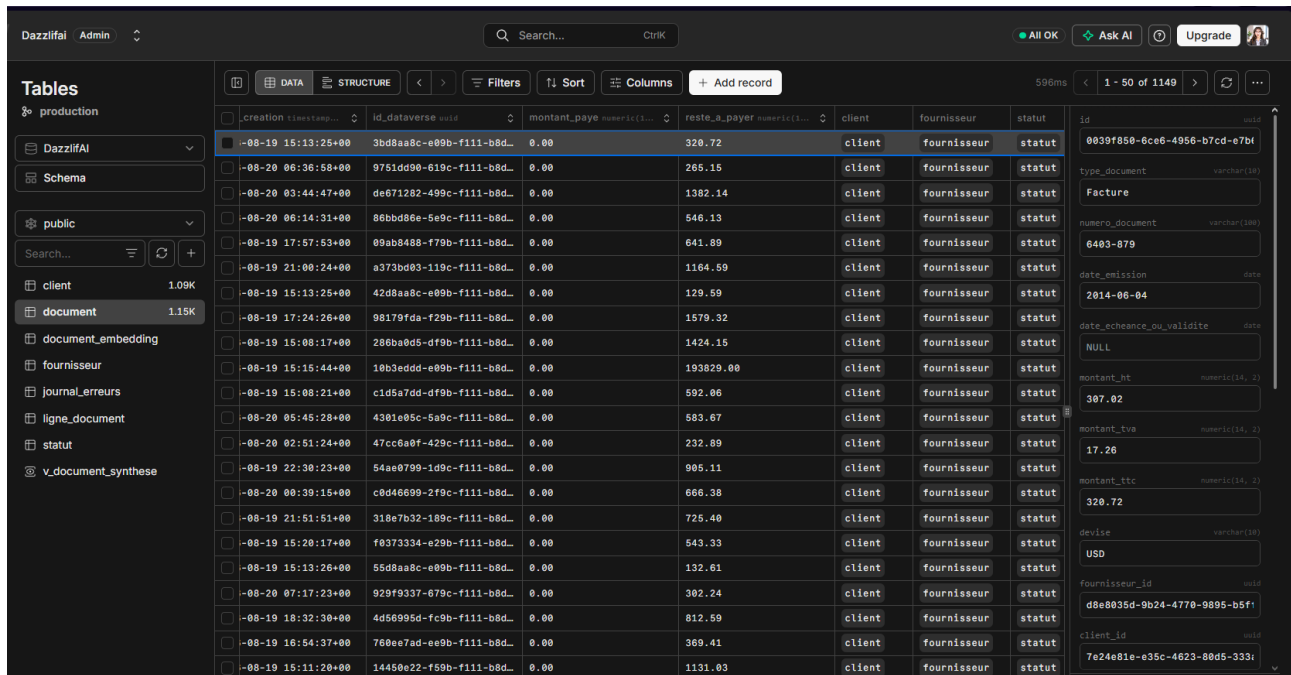


Figure 4.7: Flux *Synchronisation_Paiement* : déclenchement sur modification, récupération et mise à jour de la ligne Postgres, puis choix du statut de paiement (*Update a row*, *Update a row 1*, *Update a row 2*) selon une double condition imbriquée.

La colonne `reste_a_payer` n'est jamais recalculée par ce flux : elle est définie comme colonne générée directement en SQL, ce qui garantit sa cohérence quelle que soit la manière dont `montant_paye` est modifié, plutôt que de dupliquer cette règle de calcul dans Power Automate.



creation_timestamp	id_dataverse_uuid	montant_paye	reste_a_payer	client	fournisseur	statut
-08-19 15:13:25+00	3bd8aa8c-e09b-f111-b8d...	0.00	320.72	client	fournisseur	statut
-08-20 06:36:58+00	9751dd90-619c-f111-b8d...	0.00	265.15	client	fournisseur	statut
-08-20 03:44:47+00	de671282-499c-f111-b8d...	0.00	1382.14	client	fournisseur	statut
-08-20 06:14:31+00	86bbd86e-5e9c-f111-b8d...	0.00	546.13	client	fournisseur	statut
-08-19 17:57:53+00	09ab8488-f79b-f111-b8d...	0.00	641.89	client	fournisseur	statut
-08-19 21:00:24+00	a373bd03-119c-f111-b8d...	0.00	1164.59	client	fournisseur	statut
-08-19 15:13:25+00	42d8aa8c-e09b-f111-b8d...	0.00	129.59	client	fournisseur	statut
-08-19 17:24:26+00	98179fda-f29b-f111-b8d...	0.00	1579.32	client	fournisseur	statut
-08-19 15:08:17+00	286ba0d5-df9b-f111-b8d...	0.00	1424.15	client	fournisseur	statut
-08-19 15:15:44+00	10b3eddd-e09b-f111-b8d...	0.00	193829.00	client	fournisseur	statut
-08-19 15:08:21+00	c1d5a7dd-df9b-f111-b8d...	0.00	592.06	client	fournisseur	statut
-08-20 05:45:28+00	4301e05c-5a9c-f111-b8d...	0.00	583.67	client	fournisseur	statut
-08-20 02:51:24+00	47cc6a0f-429c-f111-b8d...	0.00	232.89	client	fournisseur	statut
-08-19 22:30:23+00	54ae0799-1d9c-f111-b8d...	0.00	905.11	client	fournisseur	statut
-08-20 00:39:15+00	c0d46699-2f9c-f111-b8d...	0.00	666.38	client	fournisseur	statut
-08-19 21:51:51+00	318e7b32-189c-f111-b8d...	0.00	725.40	client	fournisseur	statut
-08-19 15:20:17+00	f0373334-e29b-f111-b8d...	0.00	543.33	client	fournisseur	statut
-08-19 15:13:26+00	55d8aa8c-e09b-f111-b8d...	0.00	132.61	client	fournisseur	statut
-08-20 07:17:23+00	929f9337-679c-f111-b8d...	0.00	302.24	client	fournisseur	statut
-08-19 18:32:30+00	4d5e995d-fc9b-f111-b8d...	0.00	812.59	client	fournisseur	statut
-08-19 16:54:37+00	760ee7ad-ee9b-f111-b8d...	0.00	369.41	client	fournisseur	statut
-08-19 15:11:20+00	14450e22-f59b-f111-b8d...	0.00	1131.03	client	fournisseur	statut

Figure 4.8: Table document dans Neon après synchronisation complète (1149 lignes) : id_dataverse, montant_paye et reste_a_payer renseignés pour l'ensemble des documents.

4.5 | Bugs résolus pendant l'intégration

Plusieurs problèmes concrets ont été rencontrés. Une erreur OData signalant un chemin non reconnu est apparue à cause d'un champ Owner mal mappé dans Dataverse. La correction a consisté à revoir le nom logique du champ et la manière dont Power Automate construisait la requête.

Une deuxième difficulté concernait une confusion entre un identifiant UUID et une URL OData. Le flux transmettait une valeur au mauvais format, ce qui empêchait Dataverse de résoudre correctement la ligne cible. La solution a été de séparer clairement l'identifiant brut, utilisé côté Postgres, et la référence OData attendue par Dataverse.

Enfin, une incohérence de précision monétaire a été observée. Les colonnes Currency de Dataverse arrondissaient certaines valeurs à 0 décimale, alors que Postgres stockait les montants en NUMERIC(14,2). Ce point a été corrigé en ajustant les types et en validant la précision des valeurs synchronisées.

Chapter 5

Réalisation de l'agent RAG final

Le sprint final transforme le moteur de recherche construit au chapitre 3, jusque-là limité à retourner des extraits bruts, en un agent capable de répondre en langage naturel et de produire des rapports visuels à la demande. La figure 5.1 donne une vue d'ensemble du pipeline obtenu, détaillé section par section dans ce chapitre.

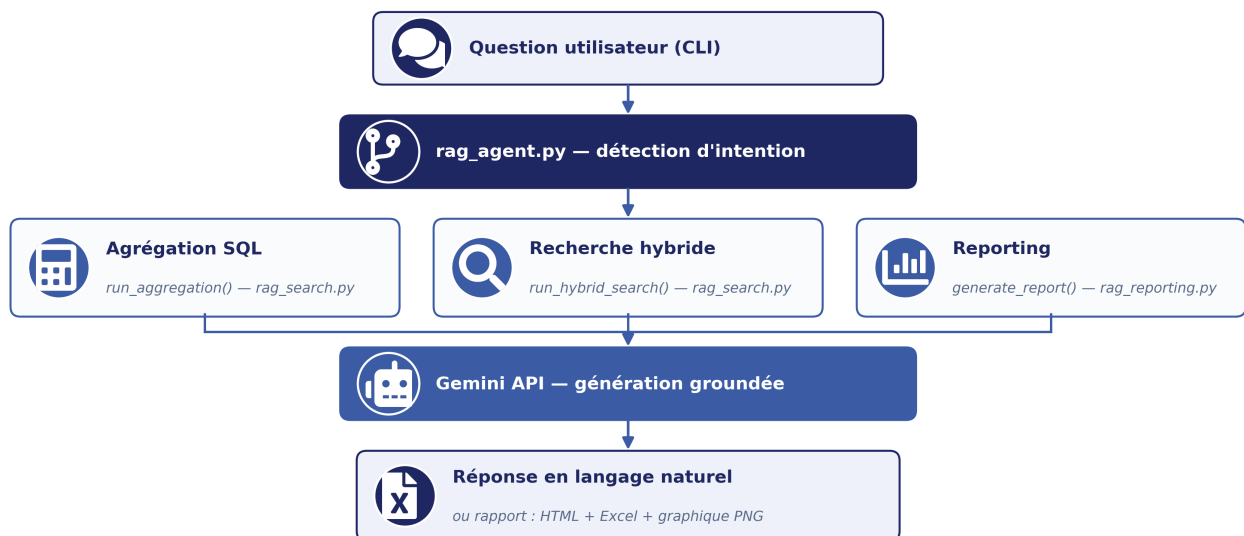


Figure 5.1: Pipeline complet : une question est routée par `rag_agent.py` vers l'un des trois traitements possibles, avant génération de la réponse finale par l'API Gemini.

5.1 | Refactoring modulaire

Le sprint final a transformé les scripts existants en une architecture plus claire et plus maintenable. Le script d'interrogation initial, monolithique et bloquant (saisie au clavier, affichage

console), a été réorganisé en cinq modules indépendants :

- `db_utils.py` : gestion de la connexion Postgres avec une logique plus résiliente (keepalive TCP et reconnexion automatique, voir chapitre 6).
- `llm_utils.py` : appels au modèle Gemini avec retry et backoff, centralisant la gestion des erreurs transitoires et des dépassements de quota.
- `rag_search.py` : recherche hybride, filtres structurés et fusion RRF, sous forme de fonctions pures retournant des structures de données plutôt que d'imprimer directement des résultats.
- `rag_reporting.py` : génération des rapports Excel, graphiques et HTML.
- `rag_agent.py` : orchestration générale entre recherche, génération et reporting — le point d'entrée unique représenté en tête de la figure 5.1.

Cette séparation a réduit le couplage entre les parties du projet. Elle rend aussi les erreurs plus faciles à localiser : une erreur de connexion, une erreur de recherche et une erreur de génération ne sont plus mélangées dans un seul script, ce qui a directement facilité le diagnostic des difficultés documentées au chapitre 6.

5.2 | Détection d'intention et routage

À la réception d'une question, `rag_agent.py` détermine, par une analyse de mots-clés, laquelle des trois branches visibles sur la figure 5.1 doit être activée : une demande de rapport (mots-clés comme *rapport*, *tableau*, *graphique*), une agrégation simple (*total*, *combien*, *reste à payer*), ou une recherche sémantique classique. Dans les deux premiers cas, le calcul est effectué directement en SQL — plus fiable et plus rapide qu'une recherche vectorielle pour ce type de question, comme évoqué au chapitre 1. Les trois branches convergent ensuite vers une même étape de génération, quel que soit le chemin emprunté.

5.3 | Génération de réponses groundées

L'agent utilise Gemini `gemini-3.6-flash` pour produire des réponses en langage naturel à partir du résultat retourné par la branche activée (extraits de documents ou tableau de chiffres). La consigne principale donnée au modèle est de répondre uniquement à partir de ce contenu récupéré. Lorsque l'information demandée n'est pas présente, l'agent doit le signaler explicitement plutôt que de la déduire.

Ce comportement est important pour un agent de facturation. Une réponse agréable mais non vérifiée serait dangereuse, car elle pourrait inventer un montant, un fournisseur ou un statut de paiement. Le RAG sert donc à encadrer la génération par des données retrouvées, conformément au principe de *grounding* présenté au chapitre 1.

5.4 | Reporting automatique

Le module `rag_reporting.py` ajoute une capacité de reporting lorsque l'intention détectée en est une. L'agent exécute alors une requête SQL avec `GROUP BY` (par fournisseur, statut, mois, type de document ou devise selon la question), puis produit trois livrables à partir du même jeu de données : un graphique en barres généré avec `matplotlib`, un classeur Excel stylisé avec `openpyxl`, et un rapport HTML auto-contenu intégrant une synthèse rédigée par le modèle de langage à partir du tableau de chiffres.

Pour garder une lecture claire même lorsque le nombre de catégories est élevé, le graphique applique une logique *Top 8 + Autres* : les huit valeurs les plus importantes sont affichées individuellement, et le reste est regroupé dans une catégorie unique plutôt que de multiplier les barres à peine visibles.

5.5 | Orchestration globale

Le rôle de `rag_agent.py` est donc de coordonner l'ensemble des composants représentés sur la figure 5.1 : il reçoit la question utilisateur, identifie la branche à activer, appelle le module correspondant, transmet le résultat au modèle de génération, puis retourne une réponse exploitable — texte simple, ou rapport accompagné de ses trois fichiers.

Cette organisation rend l'agent plus proche d'un outil utilisable que d'un simple prototype. Elle permet d'ajouter de nouvelles intentions, de nouveaux filtres ou de nouveaux formats de rapport sans réécrire tout le pipeline : chaque nouvelle capacité s'ajoute comme une branche supplémentaire après `rag_agent.py`, sans toucher aux modules existants.

Chapter 6

Difficultés rencontrées et solutions apportées

6.1 | Quota Gemini dépassé

Pendant les tests d’embeddings et de génération, le quota gratuit Gemini a été atteint. Le message d’erreur correspondait à un code 429. Cette erreur pouvait signifier deux situations différentes : un rate-limit temporaire ou un quota journalier réellement épuisé.

La solution a été d’ajouter dans `llm_utils.py` une gestion plus fine du retry et du backoff. Les erreurs transitoires sont relancées après attente progressive, tandis que les erreurs de quota journalier sont identifiées pour éviter de répéter inutilement des appels voués à échouer.

6.2 | Dépréciation du modèle

Le projet a aussi été touché par la dépréciation du modèle `gemini-2.5-flash`. Cette situation a montré qu’une dépendance à un service IA externe doit être isolée dans le code. Si le nom du modèle est dispersé dans plusieurs scripts, chaque changement devient fragile.

Le refactoring a donc centralisé les appels LLM afin de remplacer plus facilement le modèle par `gemini-3.6-flash`. Cette correction a rendu le projet plus maintenable.

6.3 | Problèmes de types numériques

Un bug est apparu lors du traitement des colonnes `NUMERIC` de Postgres. Python retournait certaines valeurs sous forme de `Decimal`, alors que d’autres calculs ou visualisations utilisaient

des `float`. Le mélange des deux types pouvait provoquer des erreurs ou des conversions implicites non souhaitées.

La correction a consisté à normaliser les conversions au moment opportun, surtout avant la génération de graphiques et l'export des rapports. Les montants financiers restent stockés avec une précision adaptée dans Postgres, puis sont convertis uniquement pour l'affichage ou les traitements qui l'exigent.

6.4 | Paramètres SQL dans les CTE

Une erreur plus subtile a concerné une requête SQL avec plusieurs CTE. Les paramètres positionnels devenaient difficiles à maintenir, car le même filtre pouvait être utilisé dans plusieurs parties de la requête. Une modification locale risquait alors de casser l'ordre des paramètres.

La solution a été de passer à des paramètres nommés. Cette correction a rendu les requêtes plus lisibles et plus sûres, en particulier dans le module de reporting.

6.5 | Erreurs Dataverse et Power Automate

Dans l'intégration Power Platform, l'erreur `odata.UnrecognizedPathException` a été provoquée par un mauvais mapping du champ `Owner`. Une autre erreur venait d'une confusion entre UUID et URL OData. Ces problèmes ont été résolus en séparant les identifiants internes Postgres des références attendues par Dataverse.

La dernière difficulté concernait la précision monétaire. Dataverse arrondissait certaines colonnes `Currency` à 0 décimale, alors que Postgres utilisait `NUMERIC(14,2)`. Cette différence a imposé une vérification des types et de la précision dans les deux systèmes.

Chapter 7

Résultats et évaluation

7.1 | Jeu de documents traité

Le jeu de test utilisé pendant le stage est stocké dans un dossier dédié compatible avec OneDrive. Il contient 1322 fichiers de factures, aux formats PDF, JPG et PNG, ce qui a permis de tester à la fois des documents numériques et des images scannées, avec la diversité de qualité et de mise en page décrite au chapitre 3. *[Répartition exacte par format et par type de document à préciser.]*

Les résultats montrent que le pipeline est capable d'extraire les informations principales, de les stocker dans Postgres, de générer des embeddings en dimension 1536 et de retrouver les documents pertinents par recherche hybride, comme détaillé au chapitre 3. À l'issue du sprint 7, l'ensemble des documents traités est synchronisé avec Dataverse et consultable depuis l'outil de facturation Power Apps (chapitre 4), avec 1149 documents actifs recensés dans les deux systèmes.

7.2 | Exemples d'interrogation de l'agent

L'agent RAG permet de poser des questions en langage naturel sur les documents traités. Les tests incluent des questions simples et des requêtes avec filtres, par exemple sur un fournisseur ou une agrégation, illustrant les deux modes de traitement présentés au chapitre 5 : recherche hybride pour les questions ouvertes, agrégation SQL directe pour les questions portant sur un total ou un dénombrement.

```

Question: Y a-t-il des factures avec des anomalies en 2023 ?

Aucun document pertinent trouvé pour cette question.

Question: Y a-t-il des factures avec statut ok en 2023 ?

RÉPONSE
=====
Oui, les 4 factures fournies dans le contexte ont le statut **OK** et ont été émises en 2023 :

1. **Document ID : ** `8262c9d6-7e49-4c87-92c1-27b1e0cffae0` (Facture n° 654123, émise le 15/01/2023)
2. **Document ID : ** `342551ff-ff2f-4212-b519-d920f4c83b91` (Facture n° 654123, émise le 15/01/2023)
3. **Document ID : ** `3df74cc8-19e1-434b-9b33-bc041ae93dd4` (Facture n° 51109329, émise le 08/11/2023)
4. **Document ID : ** `1255f3da-6977-4660-b2e3-7eea70395049` (Facture n° 51109329, émise le 08/11/2023)

```

Figure 7.1: Exemple de question simple adressée à l'agent RAG, avec réponse groundée sur le contexte récupéré.

```

Output Debug Console Ports Terminal Problems
KeyboardInterrupt
(.venv) PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis>
* History restored

PS C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Users\adali\OneDrive\Desktop\Agent RAG traitement factures et devis\.venv\Scripts\Activatepython rag_agent.py

Agent RAG - Factures et devis
Tapez votre question, ou 'exit' pour quitter.

Question: Quel est le reste à payer total pour le fournisseur Barnes, Garcia and Martin ?

RÉPONSE
=====
Le reste à payer total pour le fournisseur Barnes, Garcia and Martin s'élève à **15 334,62 €**, ce qui correspond à un ensemble de **26 factures impayées**.
Aucun montant n'a été réglé pour le moment sur ces factures.

Détail: {'nb_impayees': 26, 'total_reste_a_payer': 15334.62, 'total_paye': 0.0}

Question: 

```

Figure 7.2: Exemple de requête combinant un filtre nommé (fournisseur) et une agrégation (reste à payer).

7.3 | Rapports générés

Le module de reporting produit des rapports Excel, HTML et un graphique à partir des données Postgres, selon le mécanisme détaillé au chapitre 5. Les graphiques utilisent un regroupement *Top 8 + Autres* afin de garder une visualisation lisible lorsque le nombre de fournisseurs ou de catégories augmente. Trois exemples de rapports, portant sur trois dimensions de regroupement différentes, illustrent ce comportement.

7.3.1 Rapport 1 — Montant TTC par fournisseur

Montant TTC total par Fournisseur

Question : "Fais-moi un rapport du montant TTC par fournisseur" - Généré le 27/08/2026 à 09:03

SYNTHÈSE

Voici le rapport synthétique destiné à la direction financière : L'analyse des encours met en évidence une extrême concentration des montants, portée principalement par le fournisseur TechVision Distributors qui totalise à lui seul près de 3,2 millions d'euros TTC sur seulement deux factures. Les quatre premiers tiers regroupent l'écrasante majorité de la dette fournisseurs avec un très faible volume documentaire, contrastant fortement avec le reste du portefeuille caractérisé par une multitude de factures de faible valeur unitaire (jusqu'à 47 documents par fournisseur). Néanmoins, l'élément d'alerte le plus critique réside dans l'absence totale de règlements enregistrés dans la base, le reste à payer étant strictement égal à 100 % du montant TTC global. Une vérification immédiate du calendrier des échéances s'impose afin de prioriser le règlement des gros encours et d'éviter tout risque de pénalités ou de rupture d'approvisionnement.

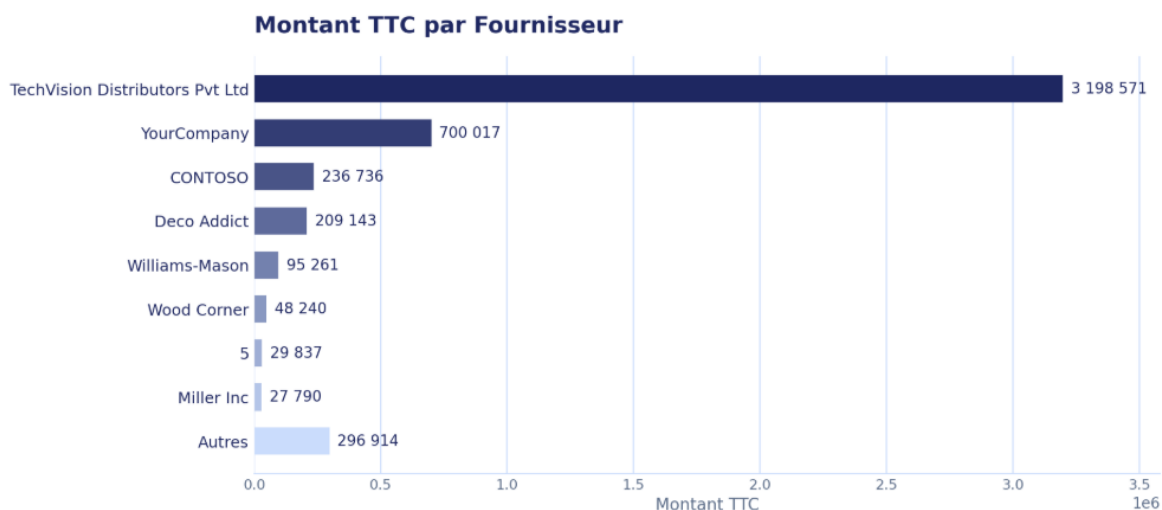


Figure 7.3: Rapport généré automatiquement : montant TTC total par fournisseur, avec synthèse rédigée par le LLM.

DÉTAIL DES DONNÉES

Fournisseur	Nombre de documents	Montant TTC	Montant payé	Reste à payer
TechVision Distributors Pvt Ltd	2.0	3 198 571	0	3 198 571
YourCompany	4.0	700 017	0	700 017
CONTOSO	2.0	236 736	0	236 736
Deco Addict	2.0	209 143	0	209 143
Williams-Mason	2.0	95 261	0	95 261
Wood Corner	2.0	48 240	0	48 240
5	47.0	29 837	0	29 837
Miller Inc	42.0	27 790	0	27 790
Duran, Morgan and Lopez	41.0	27 719	0	27 719
DSD LAD	43.0	27 048	0	27 048
Navarro, Ford and Bryan	40.0	26 370	0	26 370
White and Sons	40.0	26 207	0	26 207
RR	41.0	25 498	0	25 498
Rogers, Olson and Sanders	39.0	25 372	0	25 372
CALLLENTY	41.0	25 026	0	25 026
Berg, Ward and Wright	41.0	23 354	0	23 354
ZZ	32.0	23 104	0	23 104
HWIGH	37.0	22 600	0	22 600
SIMPLEHY	38.0	22 507	0	22 507
AGA	32.0	22 109	0	22 109

Figure 7.4: Détail complet des données du même rapport, exporté en parallèle au format Excel.

7.3.2 Rapport 2 — Reste à payer par statut

RAPPORT AUTOMATIQUE - AGENT RAG FACTURES

Reste à payer total par Statut

Question : "Fais-moi un tableau du reste à payer par statut" · Généré le 27/08/2026 à 09:07

SYNTHÈSE

L'encours restant à payer est très fortement concentré sur le statut « OK », qui représente à lui seul plus de 4,18 millions d'euros (soit près de 80 % du montant total) pour seulement 23 documents à très forte valeur unitaire. À l'inverse, les anomalies (de montant et/ou de date) engorgent le processus opérationnel en regroupant 98 % du volume global (1 299 factures), mais pour un enjeu financier plus diffus d'environ 1,14 million d'euros. Le point d'attention majeur réside dans l'absence totale de règlement enregistré à ce jour sur l'ensemble de ces dossiers. La priorité financière absolue doit donc être le recouvrement immédiat des 23 factures conformes pour libérer rapidement la quasi-totalité de la trésorerie, parallèlement à un chantier de résolution des anomalies pour solder le volume résiduel.

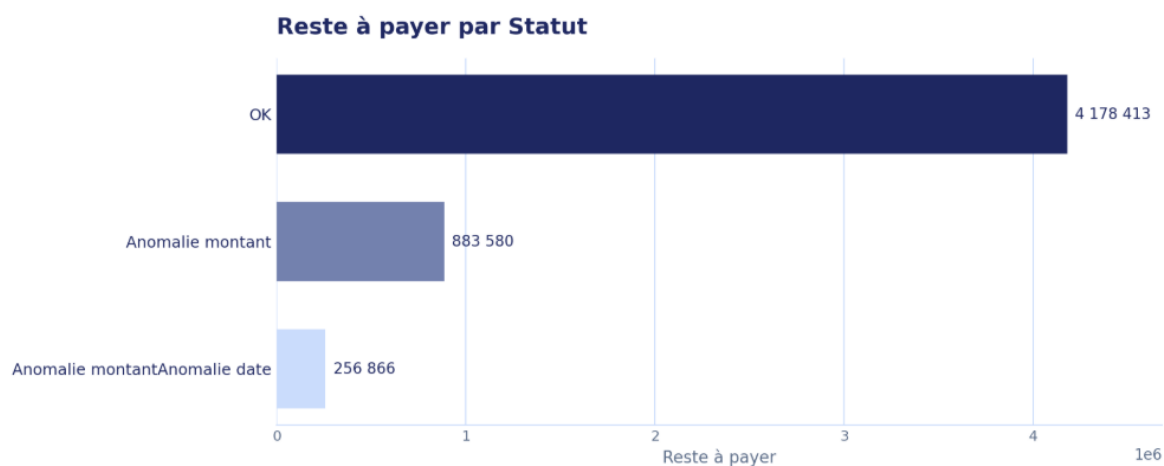


Figure 7.5: Rapport généré automatiquement : reste à payer total regroupé par statut de document.

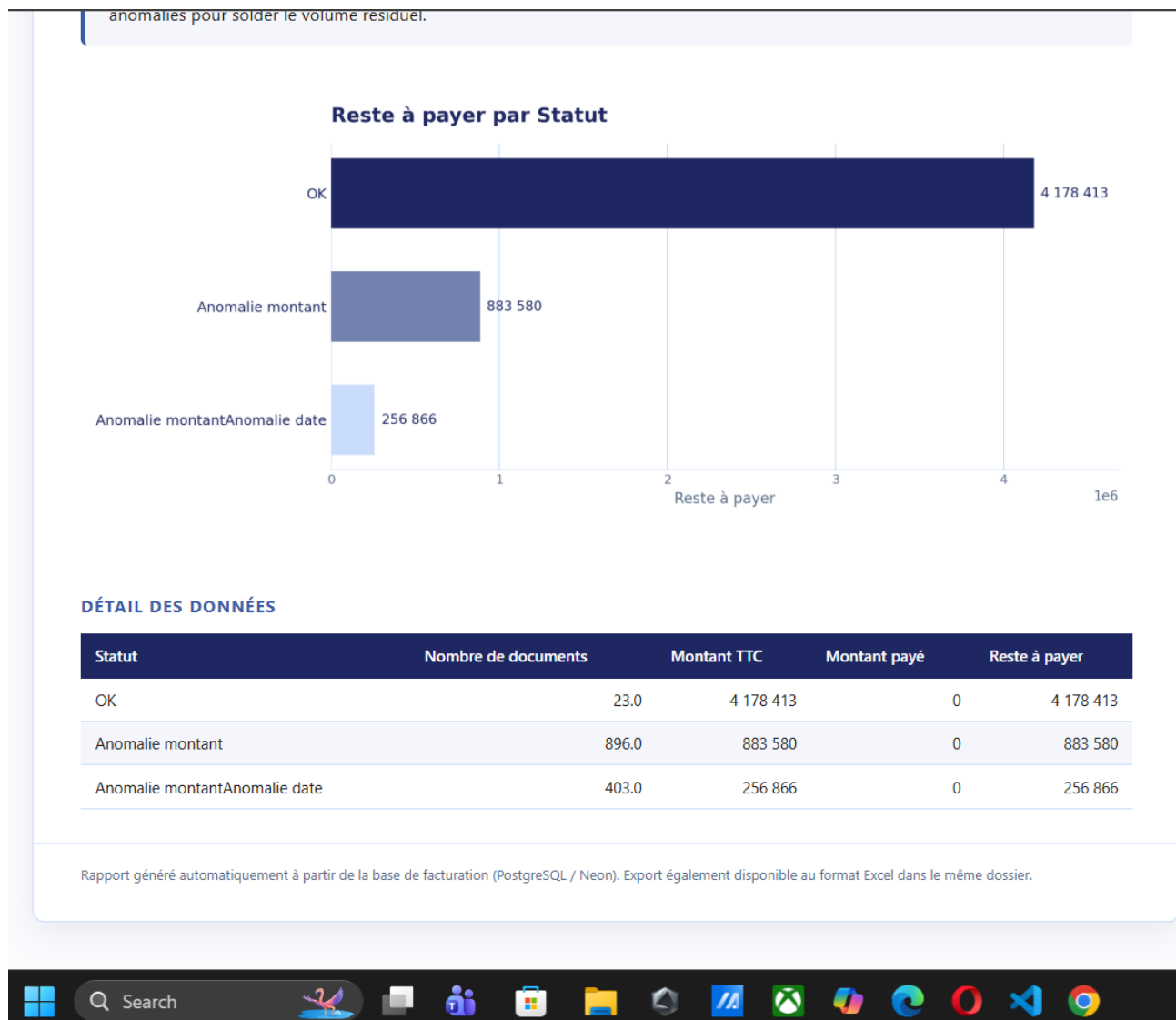


Figure 7.6: Graphique et tableau détaillé du même rapport.

7.3.3 Rapport 3 — Nombre de documents par mois



Figure 7.7: Rapport généré automatiquement : nombre de documents regroupés par mois d'émission.

DÉTAIL DES DONNÉES				
Mois	Nombre de documents	Montant TTC	Montant payé	Reste à payer
2023-11	2.0	3 198 571	0	3 198 571
2015-03	7.0	701 450	0	701 450
2015-01	6.0	259 523	0	259 523
2023-01	2.0	236 736	0	236 736
2019-04	4.0	97 358	0	97 358
None	51.0	31 640	0	31 640
2020-09	3.0	22 330	0	22 330
2022-09	6.0	6 171	0	6 171
2005-01	8.0	5 739	0	5 739
2013-12	7.0	5 699	0	5 699
2000-11	6.0	5 579	0	5 579
2014-06	11.0	5 296	0	5 296
1994-01	6.0	5 292	0	5 292
2021-10	7.0	5 262	0	5 262
2014-07	7.0	5 261	0	5 261
1999-11	7.0	5 218	0	5 218
2021-09	7.0	5 083	0	5 083
2012-09	8.0	5 040	0	5 040
1996-06	7.0	5 004	0	5 004
1996-02	5.0	4 874	0	4 874

Rapport généré automatiquement à partir de la base de facturation (PostgreSQL / Neon). Export également disponible au format Excel dans le même dossier.

Figure 7.8: Détail des données par mois, trié par montant décroissant.

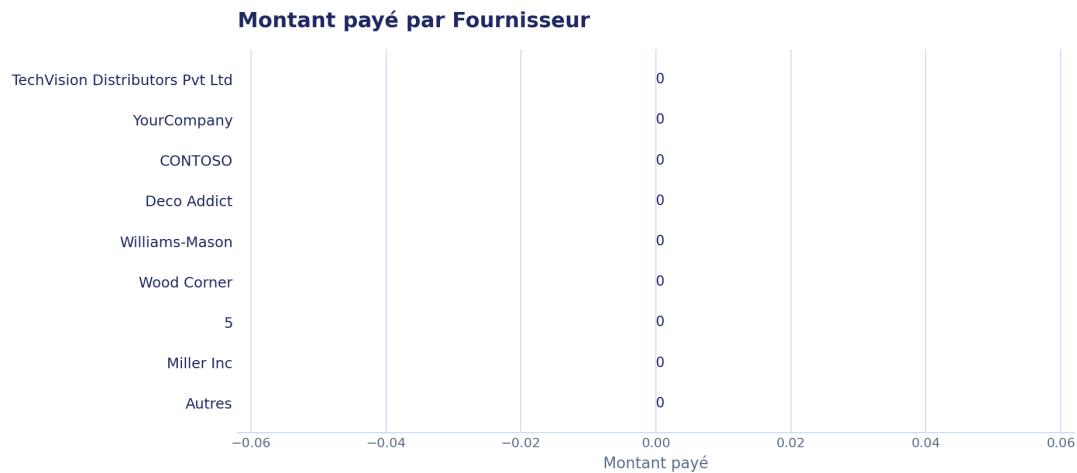


Figure 7.9: Graphique horizontal généré avec le regroupement Top 8 + Autres appliqué aux catégories principales.

7.4 | Traçabilité

Les rapports sont organisés par horodatage. Chaque génération crée un dossier `reports/<horodatage>/` contenant les trois formats produits : graphique, Excel et HTML.

Cette organisation facilite la comparaison entre plusieurs exécutions et évite d'écraser les résultats précédents, comme détaillé au chapitre 5.

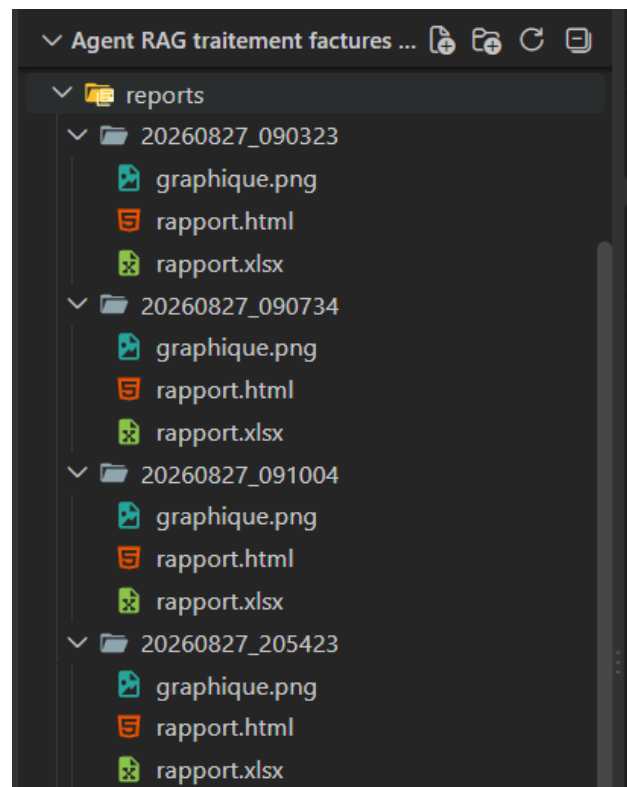


Figure 7.10: Organisation des sorties de reporting dans des dossiers horodatés.

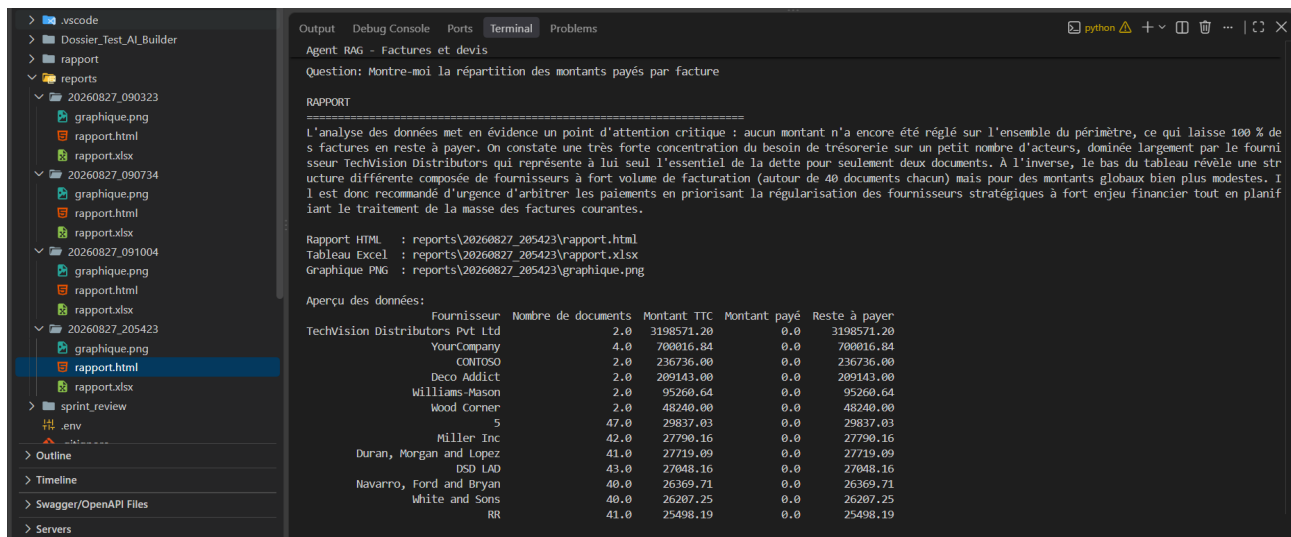


Figure 7.11: Exemple supplémentaire de rapport généré par l'agent, portant sur la répartition des montants payés par facture.

7.5 | Évaluation globale

L'évaluation globale repose sur la vérification des principales briques de la solution, depuis l'extraction des documents jusqu'à la génération des rapports. Le tableau 7.1 présente les résultats observés ainsi que leur niveau de validation à l'issue du stage. Les principaux acquis de cette évaluation peuvent être résumés comme suit :

- **Fiabilité** : les données sont contrôlées à plusieurs étapes avant leur exploitation par l'agent ou les rapports ;
- **Traçabilité** : les documents, erreurs, paiements et rapports conservent des identifiants et des horodatages vérifiables ;
- **Interopérabilité** : PostgreSQL, Dataverse, Power Apps, Power Automate et les scripts Python communiquent dans une architecture cohérente ;
- **Évolutivité** : la séparation des composants facilite l'ajout de nouveaux formats, contrôles ou indicateurs.

Table 7.1: Synthèse des résultats et de leur validation

Brique	Résultat observé	État
Extraction IDP	1322 documents traités avec scores de confiance et détection des anomalies.	Validé
Stockage	Schéma PostgreSQL normalisé et 1149 documents actifs synchronisés avec Dataverse.	Validé
Gestion des erreurs	Journalisation des incidents et alerte automatique sur les anomalies critiques.	Validé
Indexation	Embeddings de 1536 dimensions générés, stockés et contrôlés avec <code>pgvector</code> .	Validé
Recherche hybride	Fusion des résultats vectoriels et textuels par RRF, vérifiée dans Neon.	Validé
Agent RAG	Réponses générées à partir du contexte récupéré afin de limiter les hallucinations.	Validé
Reporting	Rapports Excel, HTML et graphiques produits sur plusieurs axes d'analyse.	Validé

Les tests ont également révélé des points d'attention importants, notamment les quotas LLM, la stabilité des connexions longues et les différences de types entre les outils, qui sont détaillés au chapitre 6, ainsi que des limites plus structurelles — absence de rejet effectif des doublons, détection d'intention par mots-clés — discutées au chapitre 8.

Chapter 8

Limites identifiées et recommandations

8.1 | Limites

8.1.1 Qualité variable des documents sources

Le pipeline reste sensible à la qualité du document d'entrée : une facture claire au format PDF natif donne de meilleurs résultats qu'une image floue, mal cadrée ou partiellement lisible. Les tests menés sur AI Builder (chapitre 2) ont confirmé que l'extraction reste fiable sur des documents propres, mais rien ne garantit ce même niveau de fiabilité sur l'ensemble du jeu de 1322 documents, dont la qualité de numérisation est volontairement hétérogène. C'est précisément la raison d'être des scores de confiance et des statuts d'anomalie mis en place au chapitre 3 : ils ne suppriment pas le besoin de validation humaine, ils le concentrent sur les cas réellement incertains plutôt que d'exiger une relecture systématique.

8.1.2 Dépendance aux services externes

Le projet repose sur plusieurs services cloud dont le comportement échappe au contrôle du pipeline : quotas gratuits de l'API Gemini, dépréciation d'un modèle en cours de route, ou changement de comportement d'une API tierce. Le chapitre 6 documente trois incidents concrets de cette nature (quota d'embedding épuisé, modèle **gemini-2.5-flash** retiré, erreurs 429 mal différenciées). Le code a été rendu résilient à ces incidents une fois survenus (retry, bascule de modèle via variable d'environnement), mais cette résilience reste réactive : rien dans le système actuel ne surveille ces dépendances en continu ni n'alerte avant qu'un quota ne soit réellement atteint.

8.1.3 Contraintes de la plateforme Power Platform

Dataverse et Power Automate permettent de construire rapidement une interface métier, mais imposent leurs propres contraintes : typage des colonnes, format des références OData, permissions, et précision des colonnes monétaires. Les trois bugs résolus au chapitre 4 (champ **Owner** mal mappé, confusion UUID/URL OData, arrondi de précision **Currency**) ne sont pas des erreurs de conception isolées : ce sont des manifestations directes de ces contraintes de plateforme, qui referont probablement surface à chaque nouvelle intégration Dataverse si elles ne sont pas documentées de façon centralisée.

8.1.4 Absence de déduplication effective

Le test de cohérence mené sur des copies d'un même devis (chapitre 3) a confirmé que l'extraction reste identique quel que soit le nom de fichier, mais a également révélé qu'aucun mécanisme n'empêche l'insertion de plusieurs copies d'un même document comme des enregistrements distincts en base. Sur un flux réel où les utilisateurs peuvent redéposer un fichier par erreur, cette absence de filtre peut fausser les agrégations produites par le module de reporting (chapitre 5) sans qu'aucune alerte ne soit levée.

8.1.5 Détection d'intention par mots-clés

Le routage entre agrégation SQL, recherche hybride et reporting (chapitre 5) repose entièrement sur la présence de mots-clés dans la question posée. Cette approche est simple à diagnostiquer et suffisante pour les scénarios testés durant le stage, mais elle reste fragile face à une reformulation inhabituelle : une question comme « *à combien s'élève l'ensemble des factures de mars* » ne contient aucun des mots-clés attendus (*total*, *rapport*, *combien*) et serait donc traitée comme une recherche sémantique plutôt que comme l'agrégation qu'elle exprime réellement.

8.2 | Recommandations

Le tableau 8.1 reprend chacune des limites identifiées et la recommandation correspondante, avec une priorité indicative pour une éventuelle mise en production.

Table 8.1: Recommandations associées à chaque limite identifiée

Limite	Recommandation	Priorité
Qualité variable des documents	Maintenir un seuil de confiance explicite déclenchant une file de révision manuelle	Haute
Dépendance aux services externes	Tableau de bord de supervision : documents traités, erreurs récentes, quota LLM consommé	Haute
Contraintes Power Platform	Documenter les contraintes rencontrées (mapping <code>Owner</code> , précision <code>Currency</code>) pour toute intégration Dataverse future	Moyenne
Absence de déduplication	Détecter les doublons par hash de contenu avant insertion, plutôt que par nom de fichier	Moyenne
Détection d'intention par mots-clés	Remplacer ou compléter la détection par une classification d'intention plus robuste (modèle léger ou LLM)	Moyenne
Absence de citation détaillée des sources	Afficher, pour chaque réponse, les documents utilisés et leur score de pertinence	Haute

8.2.1 Tableau de bord de supervision

Au-delà du strict monitoring des quotas, un tableau de bord de supervision donnerait une visibilité continue sur l'état du pipeline : nombre de documents traités, erreurs récentes issues du `journal_erreurs`, documents en attente de validation manuelle, et appels LLM consommés rapportés au quota disponible. C'est la recommandation la plus structurante des six : elle transforme une surveillance aujourd'hui implicite (constatée après coup, comme au chapitre 6) en un contrôle proactif.

8.2.2 Renforcement de la validation métier

La logique de validation actuelle (chapitre 3) pourrait être étendue avec des règles métier plus fines : un montant très supérieur à l'historique du fournisseur concerné, une date manifestement incohérente, ou un fournisseur jamais rencontré pourraient déclencher automatiquement un statut de révision, en complément des contrôles de cohérence déjà en place.

8.2.3 Transparence des sources citées

Enfin, l'agent RAG gagnerait à afficher systématiquement, pour chaque réponse générée, les documents effectivement utilisés, les champs consultés et leur score de pertinence RRF (déjà calculé en interne, voir chapitre 1, mais non exposé à l'utilisateur final). Cette transparence est particulièrement importante pour un outil qui manipule des informations financières : elle permet à l'utilisateur de vérifier lui-même la réponse plutôt que de devoir lui faire confiance par défaut.

Appendix A

Annexes techniques

A.1 | Organisation du jeu documentaire et du stockage

Les documents de test sont stockés dans une bibliothèque SharePoint organisée en deux dossiers principaux : **Factures** et **Devis**. Cette bibliothèque constitue la source surveillée par le déclencheur du flux Power Automate **Flux_Facture_sharepoint**, présenté au chapitre 3. La figure A.1 illustre cette organisation documentaire.

Un dossier complémentaire, nommé **Erreur**, accueille automatiquement les documents dont le traitement a échoué ou pour lesquels un champ essentiel n'a pas pu être extrait. Ces fichiers sont ainsi isolés en attente d'une vérification manuelle. Leur déplacement s'accompagne du mécanisme d'alerte par courriel décrit dans le même chapitre, afin de faciliter l'identification et le suivi des incidents.

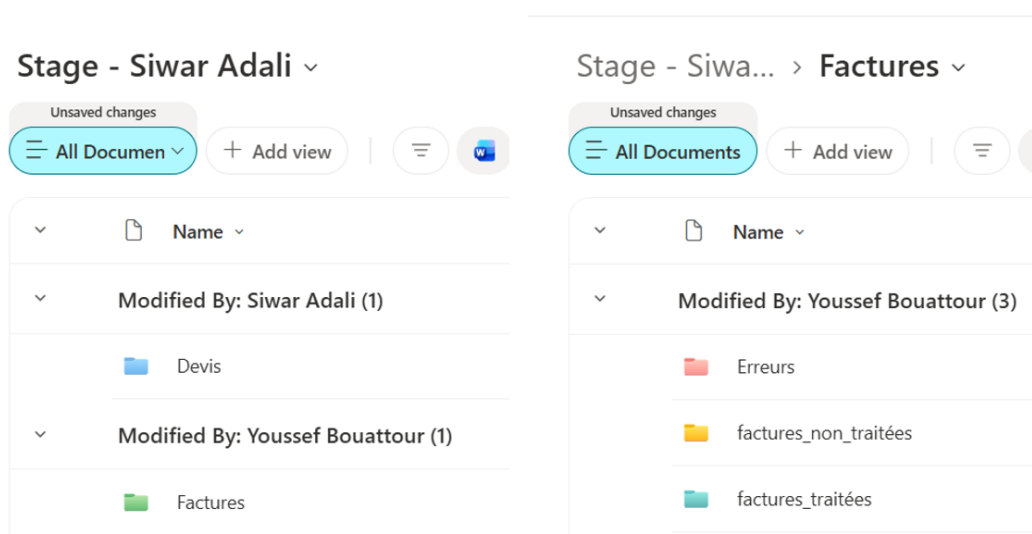


Figure A.1: Organisation de la bibliothèque SharePoint utilisée comme source du flux de traitement documentaire.

A.2 | Schéma complet de la table document

La table `document` constitue le cœur du modèle de données décrit au chapitre 3. Son schéma complet, tel qu'utilisé à l'issue du projet, est le suivant :

```
CREATE TABLE public.document (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    type_document     VARCHAR(10) NOT NULL,
    numero_document   VARCHAR(100),
    date_emission     DATE,
    date_echeance_ou_validite DATE,
    montant_ht        NUMERIC(14,2),
    montant_tva       NUMERIC(14,2),
    montant_ttc       NUMERIC(14,2),
    devise            VARCHAR(10),
    fournisseur_id    UUID,
    client_id         UUID,
    statut_id         UUID,
    confiance_montant  NUMERIC(4,3),
    confiance_fournisseur NUMERIC(4,3),
    nom_fichier_source VARCHAR(255),
    chemin_archive    TEXT,
    date_traitement   TIMESTAMPTZ NOT NULL DEFAULT now(),
    date_creation     TIMESTAMPTZ NOT NULL DEFAULT now(),
    id_dataverse      UUID,
    montant_paye      NUMERIC(14,2) DEFAULT 0,
    reste_a_payer     NUMERIC(14,2)
        GENERATED ALWAYS AS (montant_ttc - montant_paye) STORED,
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Les colonnes `id_dataverse`, `montant_paye` et `reste_a_payer` ont été ajoutées lors du sprint 7 (chapitre 4), et `updated_at` lors du sprint final, pour permettre la ré-indexation ciblée des documents modifiés (chapitre 5).

A.3 | Tables de la base

Au-delà de `document`, la base Postgres comprend les tables suivantes :

fournisseur Données d'identification des émetteurs de documents (nom, adresse, identifiant fiscal).

client Données d'identification des destinataires des documents.

statut Table de référence des statuts de traitement (*OK*, *Anomalie montant*, *Anomalie date*).

ligne_document Lignes de détail d'une facture ou d'un devis (description, quantité, prix unitaire, montant, taux de TVA).

journal_erreurs Trace des erreurs d'extraction, d'insertion ou de validation, avec fichier concerné, motif et horodatage.

document_embedding Vecteur d'embedding (VECTOR(1536)) associé à chaque document, utilisé par la recherche hybride (chapitre 3).

A.4 | Modules Python de l'agent final

Les modules développés durant le sprint final (chapitre 5) sont organisés comme suit :

db_utils.py Connexion Postgres résiliente (keepalive TCP, reconnexion automatique).

llm_utils.py Appels à l'API Gemini avec retry et backoff, distinction entre erreurs transitoires et quota journalier épuisé.

rag_search.py Recherche hybride (vecteur + full-text), fusion RRF, filtres structurés et agrégations SQL.

rag_reporting.py Génération des rapports : requêtes GROUP BY, export Excel, graphique, rapport HTML auto-contenu.

rag_agent.py Point d'entrée : détection d'intention, orchestration des modules précédents, génération de la réponse finale.

Conclusion et apprentissages personnels

Ce stage chez Dazzlifai m'a permis de développer un prototype associant automatisation documentaire, bases de données, intégration low-code et intelligence artificielle générative. À partir d'un objectif de réduction du traitement manuel des factures et devis, le projet a abouti à une chaîne capable d'extraire et de structurer les informations, de les synchroniser avec un outil de facturation, puis de les interroger en langage naturel et de produire des rapports.

Sur le plan technique, j'ai appris que la qualité d'un agent RAG dépend de l'ensemble de la chaîne de traitement, et non du seul modèle de langage. L'extraction, la validation, la recherche et les filtres conditionnent la pertinence des réponses. Les tests et les vérifications croisées avec PostgreSQL ont ainsi été essentiels pour distinguer les erreurs de données, de requêtes et de génération.

L'intégration de Power Automate, Dataverse, Power Apps et Python m'a également sensibilisée à l'importance des formats, des identifiants et de la précision numérique. Les difficultés rencontrées ont renforcé ma rigueur dans le diagnostic et la conception modulaire. Elles m'ont surtout appris qu'une solution doit être pensée pour être surveillée, corrigée et relancée après un incident, et pas uniquement pour fonctionner lors d'un premier essai.

Les résultats obtenus constituent une base fonctionnelle, mais une mise en production nécessite encore de renforcer la gestion des doublons, la compréhension des questions et la justification des réponses, ainsi que de mesurer les performances sur un corpus de référence. Ce stage a confirmé mon intérêt pour les systèmes intelligents et m'a appris à privilégier la fiabilité, la traçabilité et l'utilité réelle pour l'utilisateur. L'IA ne remplace pas la conception : elle en rend la rigueur d'autant plus importante.

Résumés / Abstracts

Résumé

Ce rapport présente un stage d'été chez Dazzlifai consacré à l'automatisation des factures et des devis. Le prototype associe extraction intelligente, contrôles de cohérence et stockage PostgreSQL. L'intégration avec Power Automate, Dataverse et Power Apps permet de consulter les documents et de suivre les paiements. Un agent de génération augmentée par la recherche (RAG) combine recherche hybride et agrégations SQL pour répondre aux questions et produire des rapports. Les tests démontrent la faisabilité du prototype, mais la gestion des doublons, la justification des réponses et l'évaluation quantitative restent à renforcer.

Mots-clés : traitement documentaire, RAG, recherche hybride, PostgreSQL, Power Platform.

Abstract

This report presents a summer internship project at Dazzlifai focused on automating invoice and quotation processing. The solution combines intelligent extraction, consistency checks and PostgreSQL storage. Integration with Power Automate, Dataverse and Power Apps enables document consultation and payment tracking. A retrieval-augmented generation (RAG) agent uses hybrid search and SQL aggregations to answer natural-language questions and generate reports. The tests demonstrate the feasibility of the prototype while highlighting the need for further improvements in duplicate handling, source attribution and quantitative evaluation before production deployment.

Keywords: document processing, RAG, hybrid search, PostgreSQL, Power Platform.

ملخص

يقدم هذا التقرير العمل المنجز خلال تدريب صيفي في شركة دازليفاي، والذي يهدف إلى أتمتة معالجة الفواتير وعروض الأسعار. يجمع الحل بين استخراج البيانات والتحقق من اتساقها وتخزينها في قاعدة بيانات منظمة، مع واجهة للاطلاع على المستندات ومتابعة المدفوعات. كما يعتمد على وكيل للتوليد المعزز بالاسترجاع يستخدم البحث الهجين وعمليات التجميع للإجابة عن الأسئلة باللغة الطبيعية وإنتاج التقارير. توضح الاختبارات إمكانية تطبيق النموذج الأولي، مع إبراز الحاجة إلى تحسين معالجة المستندات المكررة وإظهار مصادر الإجابات وقياس الأداء قبل الانتقال إلى الاستخدام الفعلي.

الكلمات المفتاحية: معالجة المستندات، الفواتير وعروض الأسعار، التوليد المعزز بالاسترجاع، البحث الهجين، قواعد البيانات، أتمتة العمليات.